

MATE-2105: Módulo 1

Regresión, optimización, regularización y clasificación

César Galindo

2026-07-10

Tabla de contenidos

Módulo 1: Regresión, optimización, regularización y clasificación	6
Propósito del módulo	7
Cómo usar este libro	8
Prerrequisitos mínimos	9
Resultados de aprendizaje	10
Calendario y entregables	11
Mapa de dependencia	12
Qué contiene el libro	13
Convenciones de notación	14
Reglas de calidad técnica	15
Regresión lineal y mínimos cuadrados	16
Una recta elegida por mínimos cuadrados	16
Del problema escalar a la matriz de diseño	17
La pérdida cuadrática en notación matricial	18
Gradiente y ecuaciones normales	19
Cómo calcular la solución	21
Proyección y diagnóstico de residuales	22
Rango, colinealidad e interpretación	23
Regreso a las subzonas de Bogotá	23
Síntesis	25
Problemas integrados	25
Respuestas seleccionadas	26
Descenso del gradiente	28
De la derivada a un algoritmo	28
La tasa decide la trayectoria	30
La Hessiana explica la estabilidad	31
Escala y condicionamiento	32
Verificar gradiente y solución	33
Costo y gradientes por subconjuntos	34
Regreso a los dos casos	34
Síntesis	35

Problemas integrados	35
Respuestas seleccionadas	36
Validación y regularización	37
Capacidad y comportamiento fuera de muestra	37
Tres funciones para tres conjuntos de datos	38
Transformaciones y filtración de información	39
Ridge: encoger direcciones inestables	40
Lasso: una penalización con esquina	42
Seleccionar sin consultar prueba	43
Regreso a las subzonas de Bogotá	43
Síntesis	44
Problemas integrados	45
Respuestas seleccionadas	46
Regresión logística	47
De una clase a una probabilidad condicional	47
La pérdida proviene del modelo Bernoulli	48
Gradiente, Hessiana y convexidad	49
La probabilidad todavía no es una decisión	51
Matriz de confusión y métricas	51
Un umbral fijado con validación	53
Calibración y alcance de las probabilidades	54
Síntesis	54
Problemas integrados	54
Respuestas seleccionadas	55
Problemas integradores de regresión y clasificación	56
Síntesis de la parte	56
Problemas integrados	56
Respuestas seleccionadas	60
Apéndice Módulo 1: formulario y respuestas seleccionadas	61
Cómo usar este apéndice	62
Notación mínima	62
Formulario mínimo	62
Patrones de código	65
Checklist antes de entregar	66
Respuestas seleccionadas	66
Guía de estudio para el Parcial 1	68
Lab 1: Regresión lineal y descenso del gradiente	69
Propósito	70
Qué debes entregar	71
Flujo recomendado	72

Lo que autocalifica el notebook	73
Parte manual	74
Criterios de calidad	75
Errores frecuentes	76
Antes de entregar	77
Lab 2: Regularización, validación y clasificación	78
Propósito	79
Qué debes entregar	80
Flujo recomendado	81
Lo que autocalifica el notebook	82
Parte manual	83
Criterios de calidad	84
Errores frecuentes	85
Antes de entregar	86
Parcial 1 autograded	87
Alcance	88
Reglas	89
Estructura	90
Qué espera el autograder	91
Parte manual	92
Preparación recomendada	93
Errores que cuestan puntos	94
Guía de preparación Parcial 1	95
Alcance	96
Formato	97
Lo que debes poder hacer	98
Errores que cuestan puntos	99

Estrategia de estudio	100
Proyecto durante el Módulo 1	101
Objetivo	102
Entregas del módulo	103
Qué debe resolver cada entrega	104
P01: lista corta	104
P02: prepropuesta	104
P03: propuesta aprobable	104
P04: baseline	104
Qué cuenta como baseline	105
Revisión rápida antes de enviar	106

Módulo 1: Regresión, optimización, regularización y clasificación

Semanas 2-6

Propósito del módulo

Este libro acompaña el primer bloque matemático fuerte del curso. La meta no es memorizar una lista de modelos, sino aprender un patrón de trabajo que se repetirá en ciencia de datos aplicada:

```
pregunta -> datos -> representación -> modelo -> pérdida  
        -> algoritmo -> métrica -> diagnóstico -> decisión
```

En cinco semanas construiremos una cadena completa:

1. regresión lineal como modelo matricial;
2. mínimos cuadrados como problema de optimización;
3. descenso del gradiente como algoritmo general;
4. validación y regularización como control de complejidad;
5. regresión logística como puente hacia clasificación.

Tesis del módulo. Un modelo no es una respuesta por sí solo. Un modelo es una hipótesis cuantitativa que debe venir con una pérdida, una métrica, una comparación contra baseline, una validación fuera de entrenamiento y una interpretación de sus limitaciones.

Cómo usar este libro

Cada capítulo tiene cuatro niveles de lectura.

Nivel	Qué hacer	Evidencia de que lo lograste
Conceptual	Leer definiciones, notas e interpretaciones	Puedes explicar el objeto sin código
Matemático	Seguir derivaciones y proposiciones	Puedes reproducir los pasos centrales
Computacional	Leer e implementar los patrones NumPy	Puedes escribir funciones pequeñas sin copiar
Aplicado	Resolver ejercicios y conectar con laboratorios	Puedes justificar una decisión técnica

No intentes estudiar el módulo solo ejecutando notebooks. El notebook muestra el cálculo en acción; el libro explica por qué ese cálculo tiene sentido.

Prerrequisitos mínimos

Antes de empezar, debes estar cómodo con:

- vectores, matrices, transpuesta y producto matricial;
- derivadas parciales básicas;
- interpretación de una función de pérdida;
- NumPy: arreglos, broadcasting, `@`, `mean`, `sum`, `reshape`;
- partición entrenamiento/prueba y noción de baseline.

Si alguno de estos puntos está débil, usa el Módulo 0 y vuelve a este libro con dos preguntas concretas: qué objeto matemático no entiendes y en qué línea de código aparece.

Resultados de aprendizaje

Al terminar el módulo deberías poder:

1. Escribir regresión lineal múltiple en forma matricial.
2. Derivar el gradiente de MSE y la ecuación normal.
3. Implementar mínimos cuadrados y descenso del gradiente con NumPy.
4. Diagnosticar curvas de pérdida y problemas de escala.
5. Explicar condicionamiento y por qué afecta la optimización.
6. Construir features polinomiales sin mezclar entrenamiento y prueba.
7. Distinguir underfitting, overfitting, error de entrenamiento y error de validación.
8. Formular Ridge y Lasso como regularización de un problema de mínimos cuadrados.
9. Ajustar un modelo logístico para clasificación binaria.
10. Elegir métricas y umbrales según el costo de falsos positivos y falsos negativos.
11. Preparar un baseline de proyecto con partición, métrica, modelo y conclusión.

Calendario y entregables

Semana	Fechas	Foco del libro	Evidencia esperada
2	2026-08-11 a 2026-08-13	Cap. 1: regresión lineal	Lista corta de datasets y Lab 1 iniciado
3	2026-08-18 a 2026-08-20	Cap. 2: gradiente	Lab 1 entregado y prepropuesta
4	2026-08-25 a 2026-08-27	Cap. 3: validación	Propuesta aprobable y Lab 2 iniciado
5	2026-09-01 a 2026-09-03	Cap. 3: regularización	Lab 2 entregado
6	2026-09-08 a 2026-09-10	Cap. 4: logística	Parcial 1 y baseline de proyecto

Mapa de dependencia

```
matrices y vectores
-> regresión lineal
  -> mínimos cuadrados
    -> gradiente
      -> descenso del gradiente
        -> escalamiento y condicionamiento
          -> validación
            -> regularización
              -> regresión logística y métricas
```

Si te bloqueas en un capítulo, busca el nodo anterior en el mapa. Por ejemplo: si Ridge parece opaco, vuelve a mínimos cuadrados; si logística parece opaca, vuelve a pérdida, gradiente y validación.

Qué contiene el libro

Parte	Uso
Capítulo 1	Regresión lineal, matrices, MSE, ecuación normal e interpretación
Capítulo 2	Descenso del gradiente, escalamiento, tasa de aprendizaje y condicionamiento
Capítulo 3	Feature engineering, validación, Ridge, Lasso y selección de modelo
Capítulo 4	Regresión logística, log-loss, métricas, umbrales y decisión
Ejercicios integradores	Banco amplio de problemas para estudiar y preparar el parcial
Formulario y respuestas	Fórmulas centrales, checklist y respuestas seleccionadas
Lab 1	Entrega autograded sobre regresión y gradiente
Lab 2	Entrega autograded sobre regularización y clasificación
Parcial 1	Parcial autograded del módulo

Convenciones de notación

Símbolo	Significado
n	número de observaciones
d	número de variables originales
p	número de columnas de la matriz de diseño, usualmente incluyendo intercepto
X	matriz de diseño en $\mathbb{R}^{n \times p}$
y	vector objetivo en $\mathbb{R}^n$
θ	vector de parámetros en $\mathbb{R}^p$
$\hat{y}$	vector de predicciones
r	vector de residuales $y - \hat{y}$
$J(\theta)$	función de pérdida
λ	intensidad de regularización

En el código usaremos nombres legibles:

```
X_train, X_val, X_test
y_train, y_val, y_test
theta
y_pred
```

Reglas de calidad técnica

Antes de entregar un resultado del módulo, verifica:

- el dataset está separado en entrenamiento y prueba;
- todo preprocesamiento se ajusta con entrenamiento;
- existe un baseline simple;
- reportas al menos una métrica adecuada;
- la conclusión distingue desempeño numérico e interpretación;
- el notebook corre desde cero sin depender de variables ocultas;
- no hay leakage entre entrenamiento, validación y prueba.

Regla de oro. Si no puedes explicar qué minimiza tu algoritmo, qué compara tu métrica y qué decisión habilita tu resultado, todavía no has terminado el análisis.

Regresión lineal y mínimos cuadrados

De una recta y sus residuales a la formulación matricial

El conjunto sintético de regresión fue generado con tres variables y una respuesta numérica. Las primeras filas son:

id	x1	x2	x3	target
reg-001	1.352	2.206	1.344	5.723
reg-002	-0.342	-0.375	1.180	4.243
reg-003	-1.743	-1.013	0.331	1.630
reg-004	0.074	-0.793	0.397	6.130
reg-005	0.536	0.705	1.965	6.755
reg-006	0.547	1.383	-0.070	3.768

Empecemos ignorando **x2** y **x3**. Queremos aproximar **target** mediante una recta

$$\hat{y}_i = \theta_0 + \theta_1 x_{i1}.$$

La recta no pasará por todos los puntos. La diferencia

$$r_i = y_i - \hat{y}_i$$

es el residual de la observación i . El problema no consiste en borrar los residuales, algo imposible en general, sino en fijar una regla para compararlos y encontrar los parámetros que optimizan esa regla.

Una recta elegida por mínimos cuadrados

Para n observaciones definimos

$$\hat{R}_n(\theta_0, \theta_1) = \frac{1}{2n} \sum_{i=1}^n (\theta_0 + \theta_1 x_{i1} - y_i)^2.$$

El factor $1/2$ simplifica las derivadas y no cambia el minimizador. Cuando se reporte el error cuadrático medio usaremos

$$\text{MSE}_n = \frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i)^2 = 2\hat{R}_n.$$

Derivar respecto del intercepto y de la pendiente produce

$$\frac{\partial \hat{R}_n}{\partial \theta_0} = \frac{1}{n} \sum_{i=1}^n (\theta_0 + \theta_1 x_{i1} - y_i),$$

$$\frac{\partial \hat{R}_n}{\partial \theta_1} = \frac{1}{n} \sum_{i=1}^n x_{i1} (\theta_0 + \theta_1 x_{i1} - y_i).$$

En un mínimo, ambas derivadas se anulan. La primera condición dice que los residuales suman cero; la segunda, que su suma ponderada por x_{i1} también es cero. Si los valores de x_{i1} no son todos iguales, al resolver las dos ecuaciones se obtiene

$$\hat{\theta}_1 = \frac{\sum_{i=1}^n (x_{i1} - \bar{x}_1)(y_i - \bar{y})}{\sum_{i=1}^n (x_{i1} - \bar{x}_1)^2}, \quad \hat{\theta}_0 = \bar{y} - \hat{\theta}_1 \bar{x}_1.$$

Esta fórmula muestra por qué la pendiente depende de la covariación entre la entrada y la respuesta, medida en relación con la variación de la entrada.

Al usar las 160 filas se obtiene

$$\hat{y} = 4.055 + 1.652x_1,$$

con MSE 2.004. El baseline que predice siempre $\bar{y} = 4.121$ tiene MSE 5.277. La variable $\mathbf{x1}$ reduce el error dentro de esta población sintética, pero la recta aún deja variación sustancial.

Una pendiente no recupera por sí sola el mecanismo generador. El coeficiente 1.652 no coincide con el valor 2.5 utilizado para generar **target**. La variable $\mathbf{x2}$ está correlacionada con $\mathbf{x1}$ y también interviene en la respuesta. Al omitirla, la pendiente de la regresión simple resume una asociación marginal diferente del coeficiente parcial del mecanismo con tres variables.

Del problema escalar a la matriz de diseño

Una regresión con varias características conserva la misma idea. Para cada observación construimos

$$\phi(x_i) = [1 \quad x_{i1} \quad \cdots \quad x_{i,p-1}]^\top \in \mathbb{R}^p.$$

La primera coordenada representa el intercepto. Al apilar las filas obtenemos

$$\mathbf{X} = \begin{bmatrix} \phi(x_1)^\top \\ \vdots \\ \phi(x_n)^\top \end{bmatrix} \in \mathbb{R}^{n \times p}, \quad \mathbf{y} = \begin{bmatrix} y_1 \\ \vdots \\ y_n \end{bmatrix} \in \mathbb{R}^n.$$

El vector de parámetros y las predicciones son

$$\theta = [\theta_0 \quad \theta_1 \quad \dots \quad \theta_{p-1}]^\top, \quad \hat{\mathbf{y}} = \mathbf{X}\theta.$$

La matriz de diseño no tiene que coincidir con las columnas originales de la tabla. Puede contener transformaciones, indicadores e interacciones. Lo que sí debe permanecer es la correspondencia entre la fila i de $\mathbf{X}$ y el valor y_i .

objeto	dimensión	interpretación
$\mathbf{X}$	$n \times p$	diseño, incluido el intercepto
θ	p	parámetros
$\mathbf{X}\theta$	n	una predicción por observación
$\mathbf{y}$	n	respuestas observadas
$\mathbf{X}^\top \mathbf{X}$	$p \times p$	productos entre columnas del diseño
$\mathbf{X}^\top \mathbf{y}$	p	productos entre columnas y respuesta

En código, la construcción explícita es:

```
import numpy as np

def add_intercept(X):
    X = np.asarray(X, dtype=float)
    if X.ndim != 2:
        raise ValueError("X debe ser una matriz bidimensional")
    return np.column_stack([np.ones(X.shape[0]), X])

X_design = add_intercept(X_raw)
y_hat = X_design @ theta
```

La comprobación de dimensiones antecede a cualquier interpretación estadística: si `X_design @ theta` no produce una predicción por fila, el objeto matemático no ha sido construido correctamente.

La pérdida cuadrática en notación matricial

El vector de residuales se define con la convención

$$\mathbf{r} = \mathbf{y} - \hat{\mathbf{y}} = \mathbf{y} - \mathbf{X}\theta.$$

La función que minimizaremos es

$$\widehat{R}_n(\theta) = \frac{1}{2n} \|\mathbf{X}\theta - \mathbf{y}\|_2^2.$$

Para el ejemplo pequeño

$$x = (0, 1, 2, 3), \quad y = (1.1, 2.9, 5.2, 6.8),$$

cada par (θ_0, θ_1) determina una recta y un valor de la pérdida.

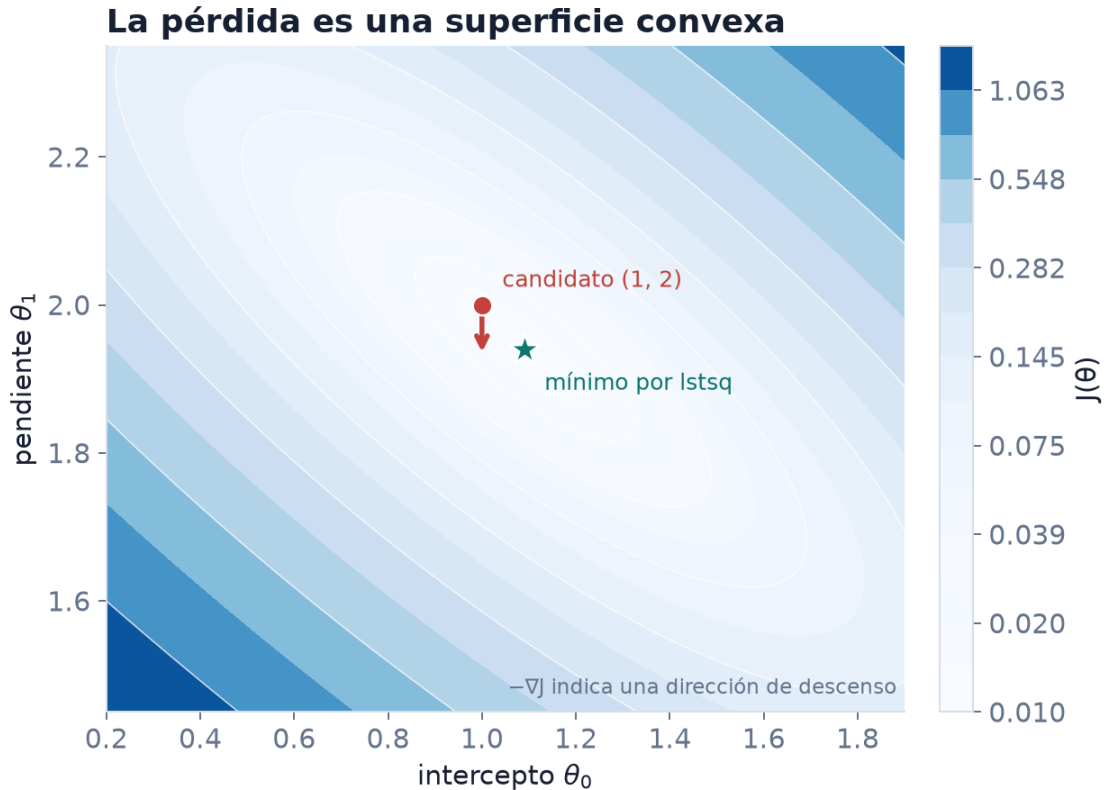


Figura 1: La pérdida cuadrática del ejemplo es convexa. El punto candidato queda cerca del mínimo calculado con `lstsq`, y el gradiente local señala cómo reducir la pérdida.

Los contornos agrupan parámetros con la misma pérdida. La convexidad permitirá caracterizar el mínimo mediante el gradiente, pero no determina cuál método numérico será más estable para calcularlo.

Gradiente y ecuaciones normales

Expandimos la forma cuadrática:

$$\widehat{R}_n(\theta) = \frac{1}{2n} (\theta^\top \mathbf{X}^\top \mathbf{X} \theta - 2\mathbf{y}^\top \mathbf{X} \theta + \mathbf{y}^\top \mathbf{y}).$$

Como $\mathbf{X}^\top \mathbf{X}$ es simétrica,

$$\nabla \widehat{R}_n(\theta) = \frac{1}{n} \mathbf{X}^\top (\mathbf{X}\theta - \mathbf{y}).$$

Una implementación que conserva exactamente esa normalización es:

```
def half_mse_loss(X, y, theta):
    error = X @ theta - y
    return 0.5 * np.mean(error ** 2)

def mse_gradient(X, y, theta):
    n = X.shape[0]
    return X.T @ (X @ theta - y) / n
```

Condición de mínimos cuadrados. Todo minimizador $\hat{\theta}$ de la pérdida cuadrática satisface

$$\mathbf{X}^\top \mathbf{X} \hat{\theta} = \mathbf{X}^\top \mathbf{y}.$$

Si $\mathbf{X}$ tiene rango columna completo, el minimizador es único.

La Hessiana es

$$\nabla^2 \widehat{R}_n(\theta) = \frac{1}{n} \mathbf{X}^\top \mathbf{X},$$

que es semidefinida positiva. Por tanto, cualquier punto donde el gradiente se anula es un mínimo global. Igualar el gradiente a cero produce las ecuaciones normales. Si $\mathbf{X}$ tiene rango columna completo, la Hessiana es definida positiva y el mínimo es único.

Hasta aquí $\mathbf{X}$ y $\mathbf{y}$ pueden verse como matrices ya observadas. Si las filas provienen de una muestra aleatoria, entonces $\hat{\theta}$ también cambia de una muestra a otra: es un estimador aleatorio. El objeto poblacional que motiva el ajuste es la media condicional $\mathbb{E}[Y \mid X = x]$; una función lineal la representa exactamente solo bajo supuestos adicionales. Las ecuaciones normales caracterizan el óptimo empírico, no demuestran por sí solas que la relación poblacional sea lineal ni que el ajuste se generalice.

Cuando $\mathbf{X}^\top \mathbf{X}$ es invertible, la identidad algebraica

$$\hat{\theta} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}$$

describe la solución. No es la receta numérica recomendada.

Cómo calcular la solución

Formar una inversa explícita añade trabajo y error de redondeo. Además, en norma 2 y bajo rango columna completo,

$$\kappa_2(\mathbf{X}^\top \mathbf{X}) = \kappa_2(\mathbf{X})^2.$$

Las ecuaciones normales elevan al cuadrado el número de condición. Una factorización QR o una SVD trabaja directamente con $\mathbf{X}$. En NumPy, `lstsq` ofrece una interfaz robusta y también devuelve el rango y los valores singulares:

```
def fit_lstsq(X, y):
    theta, residual_sum, rank, singular_values = np.linalg.lstsq(
        X, y, rcond=None
    )
    return theta, rank, singular_values
```

método	objeto principal	costo denso si $n \geq p$	observación
ecuaciones normales	$\mathbf{X}^\top \mathbf{X}$	$O(np^2 + p^3)$	empeora el condicionamiento
QR	$\mathbf{X} = \mathbf{QR}$	$O(np^2)$	estable con rango completo
SVD / <code>lstsq</code>	$\mathbf{X} = \mathbf{U}\Sigma\mathbf{V}^\top$	$O(np^2)$	revela rango y solución de norma mínima

La fórmula cerrada explica el estimador; `lstsq` calcula la solución sin formar $\mathbf{X}^\top \mathbf{X}$. Esta distinción entre identidad matemática y método numérico reaparecerá en otros modelos.

Un cálculo completo con cuatro puntos

Para los cuatro valores anteriores,

```
x = np.array([0, 1, 2, 3], dtype=float)
y = np.array([1.1, 2.9, 5.2, 6.8], dtype=float)
X = add_intercept(x.reshape(-1, 1))

theta, rank, singular_values = fit_lstsq(X, y)
y_hat = X @ theta
residual = y - y_hat
mse = np.mean(residual ** 2)
orthogonality = X.T @ residual
```

El resultado es

$$\hat{\theta} = \begin{bmatrix} 1.09 \\ 1.94 \end{bmatrix}, \quad \mathbf{r} = [0.01 \quad -0.13 \quad 0.23 \quad -0.11]^\top,$$

con MSE 0.0205. Salvo error de redondeo, $\mathbf{X}^\top \mathbf{r} = \mathbf{0}$.

Proyección y diagnóstico de residuales

La predicción $\hat{\mathbf{y}} = \mathbf{X}\hat{\theta}$ pertenece al espacio generado por las columnas de $\mathbf{X}$. Las ecuaciones normales equivalen a

$$\mathbf{X}^\top (\mathbf{y} - \hat{\mathbf{y}}) = \mathbf{0}.$$

El residual queda perpendicular al espacio columna.

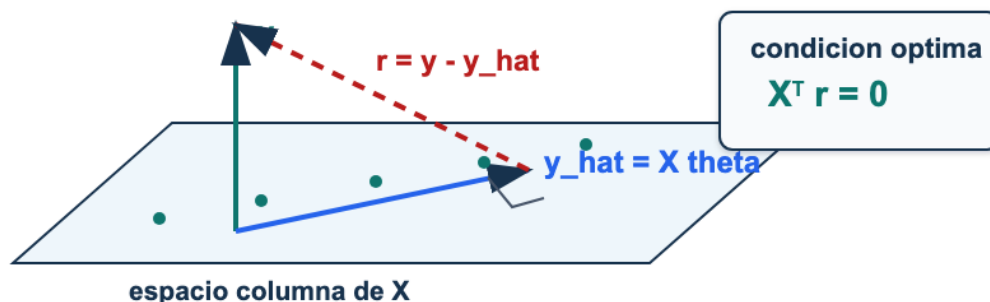


Figura 2: Geometría de mínimos cuadrados: las predicciones forman una proyección de $\mathbf{y}$ sobre el espacio columna de $\mathbf{X}$; el residual queda perpendicular a ese espacio.

La ortogonalidad es una propiedad algebraica de los datos utilizados en el ajuste. No implica que los residuales sean independientes, que tengan varianza constante ni que carezcan de estructura no lineal. Un gráfico de residual contra predicción puede revelar curvatura, dispersión creciente u observaciones influyentes incluso cuando $\mathbf{X}^\top \mathbf{r}$ es numéricamente cero.

Con las tres variables sintéticas, `lstsq` produce

$$\hat{\theta} = [4.043 \quad 2.491 \quad -1.201 \quad 0.715]^\top$$

y MSE 0.287. Los valores están cerca del mecanismo generador $(4, 2.5, -1.2, 0.7)$ y mejoran tanto la recta con `x1` como el baseline. Esto es posible porque el caso sintético conserva las variables utilizadas para generar la respuesta; no debe esperarse la misma coincidencia en datos observacionales.

Rango, colinealidad e interpretación

Si una columna de $\mathbf{X}$ es combinación lineal de otras, el rango es menor que p . Las predicciones de mínimos cuadrados siguen estando bien definidas como proyección, pero puede haber varios vectores de parámetros que producen la misma predicción.

```
x1 = np.array([1, 2, 3, 4], dtype=float)
x2 = 2 * x1
X = np.column_stack([np.ones_like(x1), x1, x2])

assert np.linalg.matrix_rank(X) < X.shape[1]
```

Con colinealidad aproximada, la solución puede ser única y, sin embargo, muy sensible a pequeñas variaciones de los datos. Coeficientes grandes de signos opuestos, cambios bruscos entre muestras y un número de condición alto son señales para revisar.

En el modelo

$$\hat{y} = \theta_0 + \theta_1 x_1 + \dots + \theta_{p-1} x_{p-1},$$

θ_j mide el cambio de la predicción cuando x_j aumenta una unidad y las otras columnas del diseño permanecen fijas. La afirmación depende de la escala y de las transformaciones. No es una afirmación causal: mantener fijas columnas correlacionadas puede describir una comparación rara o incluso imposible en la población.

Regreso a las subzonas de Bogotá

En las 32 subzonas, `estrato_promedio` y `precio_reportado` presentan una asociación positiva.

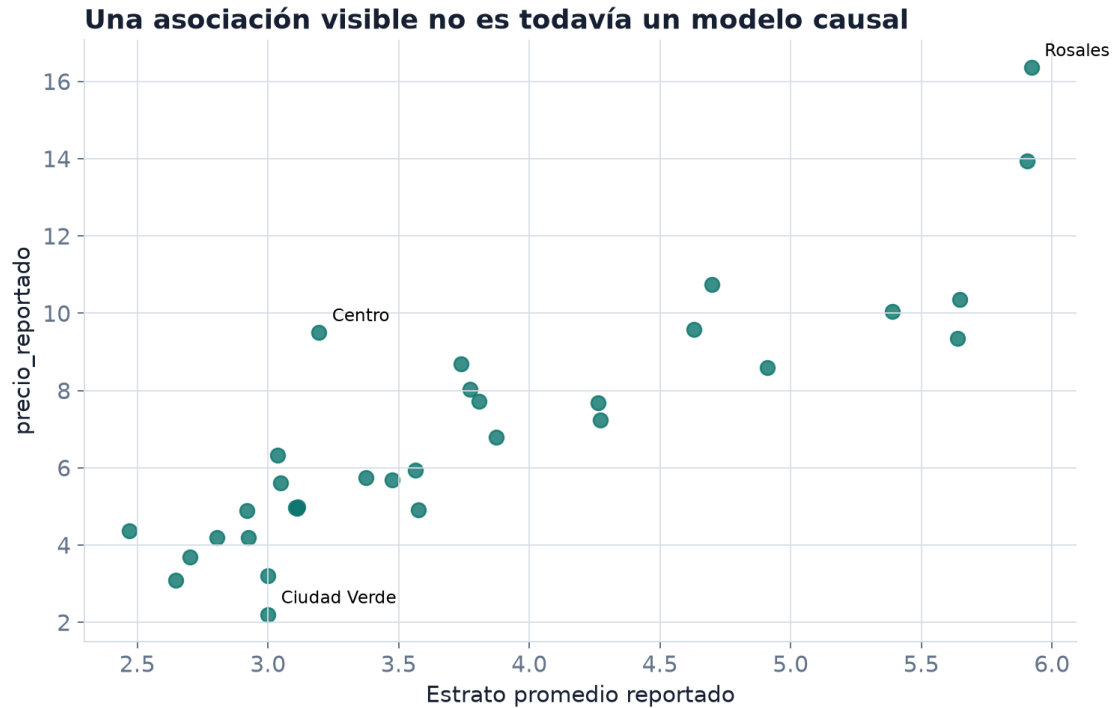
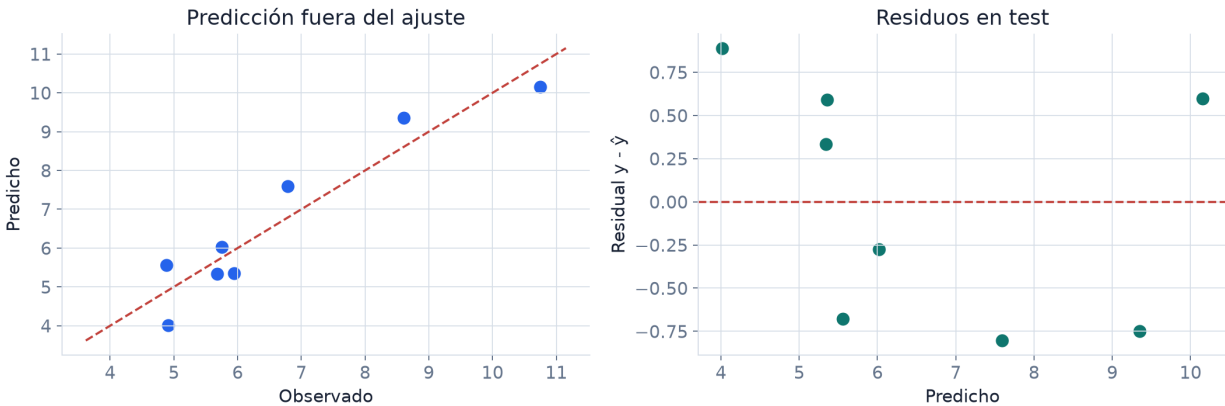


Figura 3: Estrato promedio y precio reportado presentan una asociación positiva en las 32 subzonas, pero la gráfica no identifica una relación causal.

Un ajuste con intercepto y las cinco columnas numéricas obtiene MSE de entrenamiento 1.077, frente a 9.720 para el baseline constante. También tiene un número de condición cercano a 1237, influido por escalas muy diferentes y por la relación entre promedios de área, alcobas, baños y parqueaderos.

Los MSE anteriores describen el ajuste de las 32 filas. Para obtener una primera comprobación fuera del ajuste, reservamos ocho subzonas con una semilla fijada. En esa partición, el MAE del baseline es 1.723 y el del modelo lineal es 0.615. La gráfica muestra cada predicción y el residual correspondiente.

Partición fija: MAE baseline 1.723; modelo 0.615



Ejemplo docente con 32 filas agregadas; no constituye validación operativa.

Figura 4: En una partición fija de ocho subzonas, el modelo supera el baseline; las predicciones y los residuos muestran dónde se concentra el error.

Una partición no basta para establecer que la mejora sea estable. El capítulo de validación repetirá la comparación sin escoger la semilla por su resultado. Aun entonces, una fila seguirá representando una subzona y la unidad de `precio_reportado` seguirá sin estar documentada. El álgebra no completa la información que el archivo no contiene.

Síntesis

Mínimos cuadrados comienza con residuales y una regla para agregarlos. En una regresión de una variable, las condiciones de optimalidad producen una pendiente determinada por covariación y un intercepto que hace cero el residual medio. La matriz de diseño extiende el mismo cálculo a varias características.

La pérdida $\|\mathbf{X}\theta - \mathbf{y}\|^2 / (2n)$ tiene gradiente $\mathbf{X}^\top (\mathbf{X}\theta - \mathbf{y}) / n$. Sus puntos críticos satisfacen las ecuaciones normales, y las predicciones son la proyección de $\mathbf{y}$ sobre el espacio columna de $\mathbf{X}$. Para calcular el ajuste se prefiere `lstsq` a formar una inversa o a construir $\mathbf{X}^\top \mathbf{X}$ sin necesidad.

La ortogonalidad residual certifica que el problema algebraico fue resuelto; no certifica que la forma lineal sea adecuada ni que el modelo generalice. El baseline, los gráficos de residuales, el rango y la documentación de las variables responden preguntas distintas y complementarias.

Problemas integrados

1. Una recta antes de una matriz

Considere los puntos $(0, 2)$, $(1, 2)$ y $(2, 5)$.

1. Calcule $\bar{x}$, $\bar{y}$, $\hat{\theta}_0$ y $\hat{\theta}_1$ mediante las fórmulas escalares.

2. Obtenga las predicciones, los residuales y el MSE.
3. Verifique las dos condiciones de optimalidad para los residuales.
4. Construya $\mathbf{X}$ con intercepto y reproduzca la solución con `lstsq`.
5. Compare el MSE con el baseline constante y escriba qué aporta la entrada.

2. Recuperar un mecanismo conocido

Use `regresion_lineal_sintetica.csv`.

1. Ajuste primero `target ~ x1` y luego `target ~ x1 + x2 + x3`.
2. Compare coeficientes, MSE y ortogonalidad residual.
3. Explique por qué la pendiente de `x1` cambia entre los dos diseños.
4. Añada la columna `x4 = 2*x1`, vuelva a ajustar y examine rango y valores singulares.
5. Construya dos vectores de parámetros distintos que produzcan las mismas predicciones en el diseño de rango deficiente.

3. Fórmula algebraica y algoritmo numérico

Construya matrices sintéticas con números de condición aproximadamente 10^1 , 10^3 y 10^6 .

1. Compruebe numéricamente la relación entre $\kappa_2(\mathbf{X})$ y $\kappa_2(\mathbf{X}^\top \mathbf{X})$.
2. Compare `np.linalg.inv`, `np.linalg.solve` y `np.linalg.lstsq`.
3. Mida el residual de las ecuaciones normales y el error en los coeficientes respecto de una solución conocida.
4. Explique por qué una identidad exacta puede dar lugar a implementaciones con comportamientos numéricos diferentes.

4. Un ajuste agregado en Bogotá

Use `vivienda_nueva_bogota_subzonas.csv`.

1. Defina la matriz de diseño con intercepto y las cinco columnas numéricas.
2. Reporte rango, valores singulares, número de condición, coeficientes y MSE de entrenamiento.
3. Compare con el baseline de la media y grafique residuales contra predicciones.
4. Examine cómo cambian los coeficientes después de estandarizar las cinco características sin modificar el intercepto.
5. Redacte una conclusión sobre las 32 subzonas que distinga ajuste, interpretación de coeficientes y alcance de los datos.

Respuestas seleccionadas

Problema 1. Se tiene $\bar{x} = 1$ y $\bar{y} = 3$. La pendiente es

$$\hat{\theta}_1 = \frac{(-1)(-1) + 0(-1) + 1(2)}{(-1)^2 + 0^2 + 1^2} = \frac{3}{2},$$

y el intercepto es $\hat{\theta}_0 = 1.5$. Las predicciones son $(1.5, 3, 4.5)$, los residuales son $(0.5, -1, 0.5)$ y el MSE es 0.5. Tanto la suma de residuales como $\sum_i x_i r_i$ son cero.

Problema 2. Con una sola variable se obtiene aproximadamente $(4.055, 1.652)$ y MSE 2.004. Con las tres variables se obtiene $(4.043, 2.491, -1.201, 0.715)$ y MSE 0.287. La diferencia no muestra que el primer algoritmo haya fallado: muestra que los dos modelos responden a diseños distintos.

Descenso del gradiente

Iteraciones, estabilidad y geometría de la optimización

En el capítulo anterior, `np.linalg.lstsq` calculó directamente el minimizador de

$$\widehat{R}_n(\theta) = \frac{1}{2n} \|\mathbf{X}\theta - \mathbf{y}\|_2^2.$$

Supongamos ahora que solo podemos evaluar su gradiente. Para tres observaciones con $x = (-1, 0, 1)$, $y = (1, 3, 5)$ e intercepto, comenzamos en $\theta^{(0)} = (0, 0)$. El gradiente inicial es

$$\nabla \widehat{R}_n(\theta^{(0)}) = \frac{1}{3} \mathbf{X}^\top (\mathbf{X}\theta^{(0)} - \mathbf{y}) = \begin{bmatrix} -3 \\ -4/3 \end{bmatrix}.$$

Moverse una distancia $\alpha = 0.2$ en la dirección opuesta produce

$$\theta^{(1)} = \theta^{(0)} - 0.2 \nabla \widehat{R}_n(\theta^{(0)}) = \begin{bmatrix} 0.6 \\ 0.267 \end{bmatrix}.$$

La pérdida baja de 5.833 a 3.881. No hemos resuelto un sistema ni formado una inversa: usamos información local para construir una secuencia de parámetros.

De la derivada a un algoritmo

Para una función diferenciable $J : \mathbb{R}^p \rightarrow \mathbb{R}$, el gradiente $\nabla J(\theta)$ señala la dirección de mayor crecimiento local. La aproximación de Taylor

$$J(\theta + h) \approx J(\theta) + \nabla J(\theta)^\top h$$

muestra que, entre desplazamientos de norma fija y suficientemente pequeña, la elección h opuesta al gradiente produce la mayor disminución de primer orden. El descenso del gradiente repite

$$\theta^{(t+1)} = \theta^{(t)} - \alpha \nabla J(\theta^{(t)}),$$

donde $\alpha > 0$ es la tasa de aprendizaje.

Para mínimos cuadrados,

$$\nabla \widehat{R}_n(\theta) = \frac{1}{n} \mathbf{X}^\top (\mathbf{X}\theta - \mathbf{y}).$$

La siguiente implementación guarda la pérdida inicial y la obtenida después de cada actualización. Las firmas coinciden con las utilizadas en los notebooks del módulo.

```
import numpy as np

def half_mse_loss(X, y, theta):
    error = X @ theta - y
    return 0.5 * np.mean(error ** 2)

def gradient_descent(X, y, alpha=0.1, max_iter=1000, tol=1e-8):
    n, p = X.shape
    theta = np.zeros(p, dtype=float)
    history = [half_mse_loss(X, y, theta)]

    for _ in range(max_iter):
        gradient = X.T @ (X @ theta - y) / n
        if not np.all(np.isfinite(gradient)):
            raise FloatingPointError("gradiente no finito")
        if np.linalg.norm(gradient) <= tol:
            break

        theta = theta - alpha * gradient
        loss = half_mse_loss(X, y, theta)
        if not np.isfinite(loss):
            raise FloatingPointError("pérdida no finita")
        history.append(loss)

    return theta, np.asarray(history)
```

En el ejemplo inicial, las primeras iteraciones con $\alpha = 0.2$ son:

iteración	intercepto	pendiente	pérdida	norma del gradiente
0	0.000	0.000	5.833	3.283
1	0.600	0.267	3.881	2.664
2	1.080	0.498	2.595	2.165
3	1.464	0.698	1.745	1.764
4	1.771	0.872	1.179	1.441
5	2.017	1.022	0.802	1.180

El minimizador es (3, 2). Cada columna progresa a una velocidad distinta; la secuencia todavía no ha convergido después de cinco pasos, pero la pérdida y la norma del gradiente disminuyen de manera coherente.

La tasa decide la trayectoria

La dirección del gradiente no fija la longitud del paso. En una superficie convexa, un paso pequeño puede avanzar lentamente y uno grande puede atravesar el valle y aumentar la pérdida.

Descenso del gradiente: trayectoria, escala

Cada actualización usa el gradiente local para reducir la pérdida, pero el tamaño

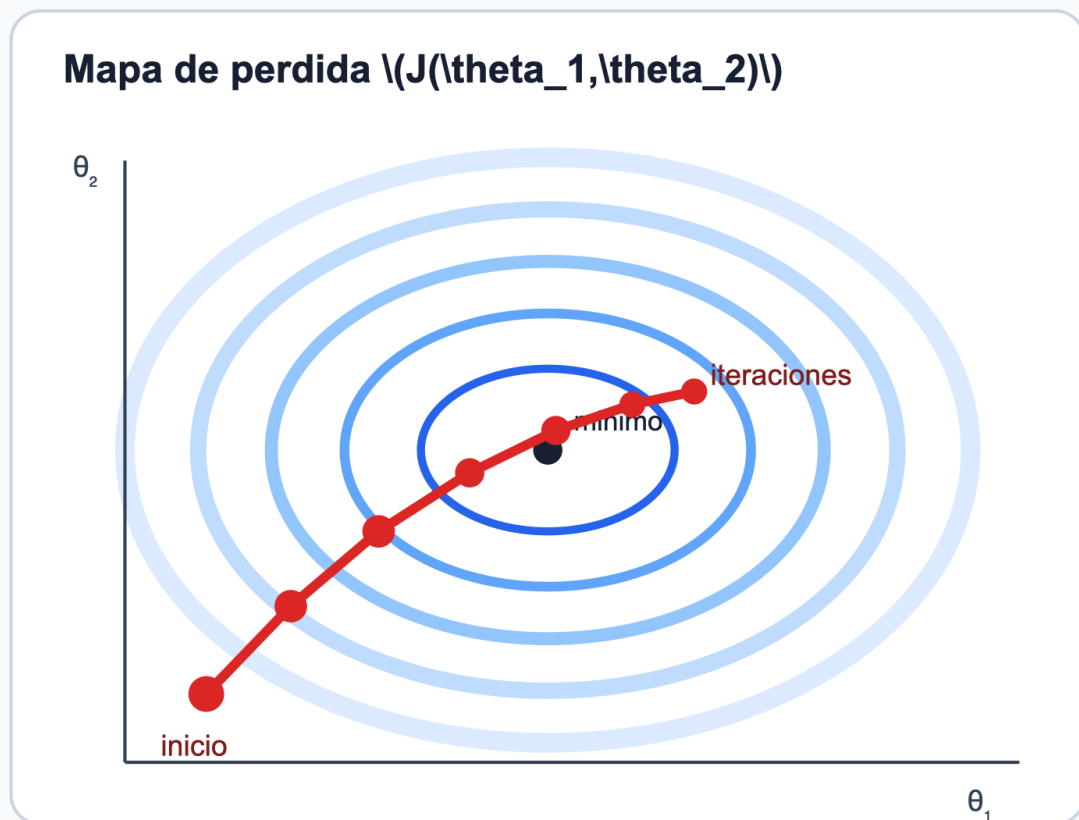


Figura 1: Descenso del gradiente: la trayectoria avanza sobre contornos de pérdida y la tasa de aprendizaje controla si el avance es lento, estable u oscilatorio.

Con el mismo ejemplo, $\alpha = 1.8$ hace oscilar el intercepto alrededor de 3, aunque la pérdida decrece. Con $\alpha = 2.2$ la pérdida comienza en 5.833 y luego toma los valores 6.770, 9.394, 13.451 y 19.352: el algoritmo diverge. Ni la convexidad de la pérdida ni la corrección del gradiente hacen estable cualquier tasa.

Una curva de pérdida debe leerse junto con la norma del gradiente y el estado de los parámetros:

comportamiento	explicación compatible	comprobación siguiente
disminución suave	paso estable	comparar con una solución de referencia
disminución muy lenta	paso pequeño o geometría alargada	revisar escala y espectro
oscilación decreciente	paso cercano al límite de estabilidad	reducir α
crecimiento sostenido nan o inf	paso fuera del intervalo estable desbordamiento tras divergencia	detener y reducir α revisar primero la historia finita
pérdida estable pero alta	familia limitada o parada prematura	separar optimización de modelado

Que la pérdida de entrenamiento converja solo responde si el algoritmo resolvió el problema empírico. No demuestra que la familia sea adecuada ni que el modelo funcione en observaciones nuevas.

La Hessiana explica la estabilidad

En mínimos cuadrados la Hessiana es constante:

$$\mathbf{H} = \nabla^2 \widehat{R}_n(\theta) = \frac{1}{n} \mathbf{X}^\top \mathbf{X}.$$

Supongamos primero que $\mathbf{X}$ tiene rango columna completo. Si θ^* es el minimizador y $e_t = \theta^{(t)} - \theta^*$, entonces

$$\begin{aligned} e_{t+1} &= e_t - \alpha \mathbf{H} e_t \\ &= (\mathbf{I} - \alpha \mathbf{H}) e_t. \end{aligned}$$

Sea $\mathbf{H} = \mathbf{Q} \mathbf{\Lambda} \mathbf{Q}^\top$ y denote sus valores propios por $0 < \lambda_1 \leq \dots \leq \lambda_p$. La componente del error en la dirección propia j se multiplica por $1 - \alpha \lambda_j$ en cada iteración. Todas las componentes se contraen exactamente cuando

$$|1 - \alpha \lambda_j| < 1 \quad \text{para todo } j,$$

o, de manera equivalente,

$$0 < \alpha < \frac{2}{\lambda_{\max}(\mathbf{H})}.$$

Convergencia en la pérdida cuadrática. Si $\mathbf{X}$ tiene rango columna completo y $0 < \alpha < 2/\lambda_{\max}(\mathbf{H})$, el descenso del gradiente con paso constante converge linealmente al único minimizador.

Si $\mathbf{H}$ es singular, las componentes iniciales en su espacio nulo no cambian y el vector de parámetros límite puede no ser único.

La tasa que equilibra las dos direcciones extremas de una cuadrática definida positiva es

$$\alpha_{\text{opt}} = \frac{2}{\lambda_{\min}(\mathbf{H}) + \lambda_{\max}(\mathbf{H})},$$

con factor de contracción máximo

$$\rho = \frac{\kappa_2(\mathbf{H}) - 1}{\kappa_2(\mathbf{H}) + 1}.$$

La fórmula explica el fenómeno, pero no es una receta general. Calcular el espectro completo puede costar tanto como resolver el problema y, fuera de una cuadrática, la Hessiana cambia con los parámetros.

Escala y condicionamiento

Una misma variable expresada en unidades distintas contiene la misma información, pero produce una geometría numérica diferente. Multipliquemos $\mathbf{x}_2$ del conjunto sintético por 1000. El diseño resultante tiene aproximadamente

$$\kappa_2(\mathbf{X}) = 1311, \quad \lambda_{\max}(\mathbf{H}) = 1.189 \times 10^6.$$

La condición teórica de estabilidad exige entonces

$$\alpha < 1.68 \times 10^{-6}.$$

Después de centrar cada característica y dividirla por su desviación estándar, sin transformar la columna de intercepto, se obtiene

$$\kappa_2(\mathbf{X}_{\text{esc}}) = 2.16, \quad \lambda_{\max}(\mathbf{H}_{\text{esc}}) = 1.657,$$

y el límite aumenta a aproximadamente 1.207. No apareció información nueva; se modificaron las coordenadas en las que el algoritmo recorre la pérdida.

La estandarización de una característica j utiliza

$$z_{ij} = \frac{x_{ij} - \mu_j}{s_j}.$$


```

mu = X_features.mean(axis=0)
scale = X_features.std(axis=0)
scale[scale == 0] = 1.0

Z = (X_features - mu) / scale
X_scaled = add_intercept(Z)

```

Cuando se evalúe fuera de la muestra de ajuste, μ_j y s_j deberán calcularse solo con entrenamiento y reutilizarse sin volverlos a estimar. El capítulo siguiente formalizará esta restricción al introducir particiones y filtración de información.

Escalar no corrige colinealidad. La estandarización puede equilibrar unidades, pero dos columnas casi redundantes continúan casi redundantes. Si un valor singular permanece cercano a cero después de escalar, el problema requiere revisar el diseño o introducir regularización.

Verificar gradiente y solución

Una implementación del gradiente debe comprobarse antes de interpretar su curva. La diferencia central en la coordenada j es

$$g_j^{\text{num}} = \frac{J(\theta + he_j) - J(\theta - he_j)}{2h}.$$

Para un h moderadamente pequeño, el vector numérico debe aproximar el gradiente analítico:

```

def finite_difference_gradient(loss, theta, h=1e-6):
    gradient = np.empty_like(theta, dtype=float)
    for j in range(theta.size):
        direction = np.zeros_like(theta, dtype=float)
        direction[j] = h
        gradient[j] = (
            loss(theta + direction) - loss(theta - direction)
        ) / (2 * h)
    return gradient

theta0 = np.zeros(X.shape[1])
analytic = X.T @ (X @ theta0 - y) / X.shape[0]
numeric = finite_difference_gradient(
    lambda theta: half_mse_loss(X, y, theta), theta0
)
assert np.allclose(analytic, numeric, rtol=1e-5, atol=1e-7)

```

Después se compara el límite del algoritmo con `lstsq` usando exactamente la misma matriz:

```

theta_ref = np.linalg.lstsq(X_scaled, y, rcond=None)[0]
theta_gd, history = gradient_descent(
    X_scaled, y, alpha=0.5, max_iter=10_000
)

parameter_gap = np.linalg.norm(theta_gd - theta_ref)
gradient_norm = np.linalg.norm(
    X_scaled.T @ (X_scaled @ theta_gd - y) / X_scaled.shape[0]
)

```

Una diferencia grande puede deberse a una tasa inestable, pocas iteraciones, un criterio de parada mal definido o un gradiente incorrecto. Comparar solo las pérdidas puede ocultar diferencias entre parámetros cuando el diseño tiene rango deficiente.

Costo y gradientes por subconjuntos

Calcular $\mathbf{X}\theta$ y después multiplicar el error por $\mathbf{X}^\top$ cuesta $O(np)$ por iteración batch. Tras T iteraciones, el costo es $O(Tnp)$, además de almacenar el diseño. Una factorización directa paga aproximadamente $O(np^2)$ una vez. El método conveniente depende de n , p , la estructura dispersa y el número de iteraciones necesario.

Para un subconjunto B de observaciones definimos

$$g_B(\theta) = \frac{1}{|B|} \sum_{i \in B} \nabla_{\theta} \ell_i(\theta).$$

Si las posiciones se seleccionan uniformemente, condicionado en el conjunto de datos,

$$\mathbb{E}_B[g_B(\theta)] = \nabla \widehat{R}_n(\theta).$$

Por tanto, el gradiente de mini-batch es un estimador condicionalmente insesgado del gradiente empírico completo. Su aleatoriedad proviene de escoger el subconjunto B , una vez fijados los datos; no debe confundirse con la variación que produciría observar otra muestra de la población. Un mini-batch reduce el costo de cada paso e introduce variación entre pasos. Esta observación prepara el uso de optimizadores estocásticos en redes neuronales; en este módulo, el algoritmo de referencia seguirá utilizando todo el conjunto.

Regreso a los dos casos

En la población sintética, el diseño con intercepto y tres características tiene número de condición 2.17. El descenso con datos sin reescalar ya es manejable y debe converger al mismo ajuste que `lstsq`. Multiplicar `x2` por 1000 no cambia las predicciones que puede representar el modelo, pero obliga a reducir drásticamente la tasa si no se estandariza.

En las subzonas de Bogotá, el diseño sin escalar tiene número de condición cercano a 1237. Parte del valor se debe a que `area_promedio` vive en una escala muy distinta de los demás promedios. Estandarizar facilita la optimización y la comparación numérica de columnas, pero no corrige la agregación por subzona, la unidad desconocida de `precio_reportado` ni la dependencia entre variables. Un algoritmo puede converger con precisión a un modelo cuya interpretación siga siendo limitada.

Síntesis

El descenso del gradiente convierte una derivada en una sucesión de parámetros. La pérdida, la norma del gradiente y la comparación con una solución de referencia permiten distinguir convergencia, oscilación, divergencia y errores de implementación.

En una pérdida cuadrática, los valores propios de $\mathbf{X}^\top \mathbf{X}/n$ determinan el intervalo estable de tasas y la velocidad relativa de las direcciones. El escalamiento puede transformar una geometría muy alargada en otra más equilibrada sin cambiar la información del problema. No elimina colinealidad ni sustituye la evaluación estadística.

El próximo capítulo mostrará un conflicto diferente: una familia más flexible puede reducir la pérdida de entrenamiento y empeorar el comportamiento fuera de muestra. Allí la optimización será una parte del procedimiento, no el criterio para escoger por sí sola el modelo final.

Problemas integrados

1. Seis pasos que pueden calcularse a mano

Use $x = (-1, 0, 1)$, $y = (1, 3, 5)$ y $\theta^{(0)} = (0, 0)$.

1. Derive el gradiente y la Hessiana.
2. Calcule seis iteraciones con $\alpha = 0.2$.
3. Repita con $\alpha = 1.8$ y $\alpha = 2.2$; compare las pérdidas.
4. Obtenga los valores propios de la Hessiana y determine el intervalo teórico de convergencia.
5. Explique cada trayectoria mediante los factores $1 - \alpha\lambda_j$.

2. El mismo dato en otra unidad

Use `regresion_lineal_sintetica.csv` y multiplique `x2` por 1000.

1. Calcule número de condición, espectro de la Hessiana y tasa máxima teórica antes de escalar.
2. Ejecute gradiente con tasas 10^{-7} , 10^{-5} y 0.1.
3. Estandarice las características, mantenga intacto el intercepto y repita.
4. Convierta los coeficientes del modelo escalado a las unidades originales y compruebe que las predicciones coinciden.
5. Distinga qué cambió en la optimización, en el modelo representable y en la interpretación de los coeficientes.

3. Auditoría de una implementación

Introduzca deliberadamente uno de estos errores: omitir $1/n$, cambiar el signo del gradiente o devolver una pérdida anterior al último parámetro.

1. Diseñe una prueba por diferencias finitas que detecte el error.
2. Compare el resultado con `lstsq`.
3. Registre pérdida, norma del gradiente y distancia entre parámetros.
4. Corrija la función sin cambiar su firma pública.
5. Escriba qué prueba habría fallado aun si la pérdida final pareciera pequeña.

4. Optimización en las subzonas

Use las cinco columnas numéricas del caso de Bogotá.

1. Compare el espectro y el número de condición antes y después de estandarizar.
2. Encuentre una tasa estable para cada representación sin seleccionar una semilla por su resultado.
3. Compare las predicciones de gradiente y `lstsq`.
4. Grafique la historia de pérdida con ejes y escala claramente identificados.
5. Redacte un diagnóstico que separe convergencia numérica, ajuste en las 32 filas y alcance sustantivo del dataset.

Respuestas seleccionadas

Problema 1. La Hessiana es

$$\mathbf{H} = \begin{bmatrix} 1 & 0 \\ 0 & 2/3 \end{bmatrix},$$

por lo que el intervalo estable es $0 < \alpha < 2$. Con $\alpha = 2.2$, la componente asociada a $\lambda = 1$ se multiplica por -1.2 y crece en magnitud; la divergencia no se debe a un mínimo local.

Problema 2. Antes de escalar, $\kappa_2(\mathbf{X}) \approx 1311$ y $\alpha_{\text{máx}} \approx 1.68 \times 10^{-6}$. Después de escalar, $\kappa_2(\mathbf{X}) \approx 2.16$ y $\alpha_{\text{máx}} \approx 1.207$. Las dos parametrizaciones pueden producir las mismas predicciones lineales; sus recorridos numéricos no son iguales.

Validación y regularización

Cómo escoger complejidad sin confundir ajuste con generalización

La variable `x1` del conjunto sintético produjo una recta con MSE cercano a 2. Podemos construir una familia mucho más flexible sin añadir nuevas mediciones:

$$\phi_d(x) = [x \ x^2 \ \dots \ x^d]^\top.$$

Dividamos las 160 filas, con semillas fijadas, en 80 para entrenamiento, 40 para validación y 40 para prueba. Ajustar polinomios por mínimos cuadrados produce:

grado	MSE entrenamiento	MSE validación
1	1.993	2.223
3	1.965	2.172
8	1.749	2.379
15	1.542	3.254

El grado 15 resuelve mejor el problema que se le pidió durante el ajuste: tiene el menor error de entrenamiento. También produce el peor resultado en las 40 observaciones que no utilizó. La optimización no falló. El criterio de selección era incompleto.

Capacidad y comportamiento fuera de muestra

Una regresión polinomial sigue siendo lineal en sus parámetros:

$$f_\theta(x) = \theta_0 + \theta_1 x + \dots + \theta_d x^d.$$

Al aumentar d , la familia contiene a las anteriores y el mínimo de la pérdida de entrenamiento no puede aumentar, salvo diferencias numéricas. Esa propiedad no se extiende al riesgo poblacional. Los parámetros de orden alto pueden adaptarse a fluctuaciones particulares de la muestra.

```
def polynomial_features_1d(x, degree):
    x = np.asarray(x, dtype=float).reshape(-1, 1)
    columns = [np.ones((x.shape[0], 1))]
    columns.extend(x ** power for power in range(1, degree + 1))
    return np.hstack(columns)
```

La transformación $x \mapsto (x, x^2, \dots, x^d)$ es determinista por fila. No consulta otras observaciones. El grado, en cambio, define una familia y debe elegirse sin utilizar la evaluación final.

Si D_n representa la muestra de ajuste y $\hat{f}_{D_n}$ la función obtenida, repetir el muestreo produce otra función. Para pérdida cuadrática y una entrada fija x , bajo condiciones usuales,

$$\begin{aligned}\mathbb{E}_{D_n, Y|x}[(Y - \hat{f}_{D_n}(x))^2] &= \text{Var}(Y | X = x) \\ &+ \text{Bias}(x)^2 \\ &+ \text{Var}_{D_n}(\hat{f}_{D_n}(x)),\end{aligned}$$

donde

$$\text{Bias}(x) = \mathbb{E}_{D_n}[\hat{f}_{D_n}(x)] - \mathbb{E}[Y | X = x].$$

Una familia demasiado rígida puede tener sesgo alto. Una familia muy flexible puede responder con intensidad a las particularidades de D_n . La regularización modifica este balance; la validación proporciona datos para compararlo.

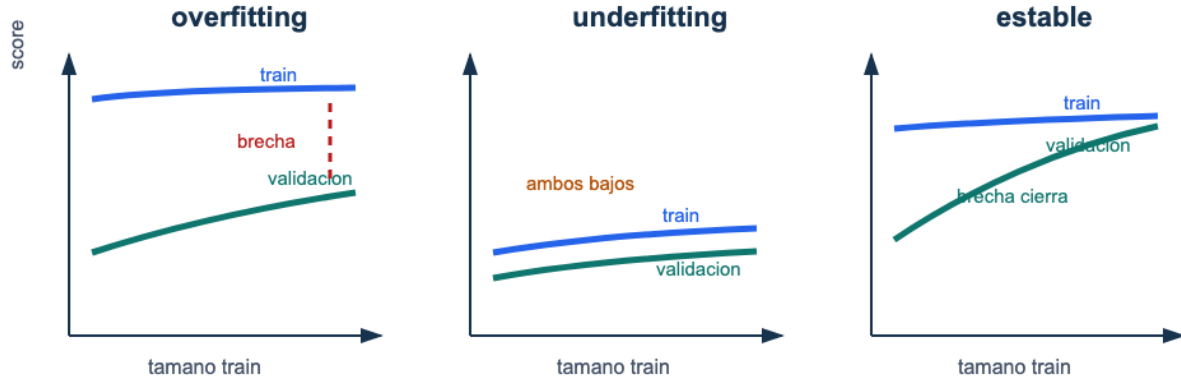


Figura 1: Curvas de aprendizaje: el sobreajuste mantiene una brecha entre entrenamiento y validación, el subajuste mantiene ambos errores altos y un caso estable reduce la brecha al aumentar los datos.

Una curva de aprendizaje es un diagnóstico, no una prueba automática de una causa. Una brecha puede sugerir sensibilidad a la muestra; errores altos en ambos conjuntos pueden sugerir una familia inadecuada, una representación pobre o una optimización incompleta.

Tres funciones para tres conjuntos de datos

Entrenamiento, validación y prueba no son tres nombres para muestras intercambiables.

conjunto	decisiones que permite
entrenamiento	ajustar parámetros y transformaciones aprendidas
validación	escoger grado, penalización y otras configuraciones
prueba	evaluar una vez el procedimiento ya fijado

Validación de una regla fijada. Sea f una función que no utilizó el conjunto de validación. Si $Z_1^{\text{val}}, \dots, Z_v^{\text{val}}$ son iid de la población P , entonces

$$\mathbb{E}[\widehat{R}_{\text{val}}(f) \mid f] = R_P(f),$$

si la pérdida es integrable.

Condicionado en f , cada pérdida $\ell(f; Z_i^{\text{val}})$ tiene esperanza $R_P(f)$. La linealidad de la esperanza aplicada al promedio da la igualdad.

Después de comparar muchos modelos y escoger el menor valor observado en esa misma validación, la función elegida ya depende de los datos de validación. El mínimo incorpora optimismo por selección. El conjunto de prueba se reserva para el procedimiento completo: transformaciones, familia, hiperparámetros, umbral y regla de entrenamiento.

Cuando hay pocas observaciones, la validación cruzada reutiliza de manera estructurada distintas partes de los datos para comparar configuraciones. No elimina la incertidumbre ni autoriza usar prueba durante el diseño. El capítulo de estrategia estudiará sus estimaciones con más detalle.

Transformaciones y filtración de información

Una transformación puede ser fija o aprender cantidades de los datos.

- $x \mapsto x^2$ se aplica a una fila sin estimar nada de las demás.
- La estandarización necesita medias y desviaciones.
- La imputación necesita reglas o valores de reemplazo.
- Una codificación puede aprender qué categorías existen.
- La selección de variables utiliza una medida calculada con respuestas.

Las cantidades aprendidas se ajustan solo con entrenamiento y se aplican sin reestimarlas a validación y prueba. De lo contrario, información reservada cambia la representación usada para entrenar.

Dos filtraciones diferentes. Calcular la media de estandarización con toda la tabla filtra información entre observaciones. Incluir una variable registrada después del resultado filtra información temporal. Una tubería de software ayuda con la primera; la segunda exige comprender cómo se produce cada columna.

Una implementación con `scikit-learn` mantiene juntos transformación y ajuste:

```

from sklearn.linear_model import Ridge
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import PolynomialFeatures, StandardScaler

model = make_pipeline(
    PolynomialFeatures(degree=3, include_bias=False),
    StandardScaler(),
    Ridge(alpha=1.0, fit_intercept=True),
)
model.fit(X_train, y_train)
validation_prediction = model.predict(X_validation)

```

El método `fit` del pipeline aprende polinomio, escalas y regresión únicamente a partir de entrenamiento. `predict` reutiliza esos objetos.

Ridge: encoger direcciones inestables

Ridge añade una penalización cuadrática a la pérdida. Con el intercepto en la primera columna de $\mathbf{X}$, definimos

$$\mathbf{D} = \text{diag}(0, 1, \dots, 1)$$

y

$$J_{\text{Ridge}}(\theta) = \frac{1}{2n} \|\mathbf{X}\theta - \mathbf{y}\|_2^2 + \frac{\lambda}{2} \theta^\top \mathbf{D} \theta.$$

El cero inicial evita penalizar el intercepto. El gradiente es

$$\nabla J_{\text{Ridge}}(\theta) = \frac{1}{n} \mathbf{X}^\top (\mathbf{X}\theta - \mathbf{y}) + \lambda \mathbf{D} \theta.$$

Igualarlo a cero produce

$$(\mathbf{X}^\top \mathbf{X} + n\lambda \mathbf{D}) \hat{\theta}_\lambda = \mathbf{X}^\top \mathbf{y}.$$

```

def ridge_closed_form(X, y, lam):
    n, p = X.shape
    penalty = np.eye(p)
    penalty[0, 0] = 0.0
    system = X.T @ X + n * lam * penalty
    return np.linalg.solve(system, X.T @ y)

```


Esta función implementa la convención del libro. `sklearn.linear_model.Ridge` minimiza

$$\|\mathbf{y} - \mathbf{X}\mathbf{w}\|_2^2 + \alpha\|\mathbf{w}\|_2^2.$$

Por tanto, para el mismo diseño y sin penalizar el intercepto, $\alpha = n\lambda$. Usar el mismo número para α y λ sin considerar la normalización no reproduce el mismo problema.

Qué ocurre en las direcciones singulares

Para estudiar los coeficientes no intercepto, supongamos que las características están centradas y escribamos $\mathbf{X} = \mathbf{U}\Sigma\mathbf{V}^\top$. Ridge actúa como

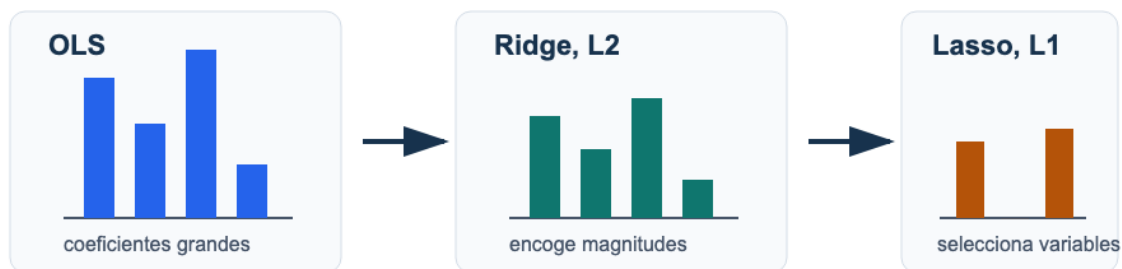
$$\hat{\theta}_\lambda = \mathbf{V}(\Sigma^\top\Sigma + n\lambda\mathbf{I})^{-1}\Sigma^\top\mathbf{U}^\top\mathbf{y}.$$

En la dirección derecha v_j , el factor

$$\frac{\sigma_j}{\sigma_j^2 + n\lambda}$$

reemplaza el factor $1/\sigma_j$ de mínimos cuadrados. Las direcciones con σ_j pequeño, que son las más sensibles, se encogen con mayor intensidad. La estabilidad se obtiene a cambio de sesgo.

Regularización: controlar complejidad sin cambiar la pregunta



Idea matemática: minimizar pérdida + penalización; validar lambda antes de decidir.

Figura 2: Regularización: mínimos cuadrados puede producir coeficientes grandes, Ridge reduce magnitudes y Lasso puede llevar coeficientes exactamente a cero.

Un λ positivo no garantiza menor riesgo. Si es demasiado grande, las predicciones se acercan a una regla casi constante y pueden perder estructura real. Su valor se selecciona con validación.

Lasso: una penalización con esquina

Lasso utiliza

$$J_{\text{Lasso}}(\theta) = \frac{1}{2n} \|\mathbf{X}\theta - \mathbf{y}\|_2^2 + \lambda \sum_{j=1}^{p-1} |\theta_j|.$$

De nuevo, el intercepto no se penaliza. La función $|t|$ no es diferenciable en cero; su subdiferencial es

$$\partial|t| = \begin{cases} \{1\}, & t > 0, \\ [-1, 1], & t = 0, \\ \{-1\}, & t < 0. \end{cases}$$

Sea $\mathbf{x}_j$ la columna j del diseño. Una solución satisface

$$0 \in \frac{1}{n} \mathbf{x}_j^\top (\mathbf{X}\hat{\theta} - \mathbf{y}) + \lambda \partial|\hat{\theta}_j|.$$

La coordenada puede quedar exactamente en cero cuando

$$\left| \frac{1}{n} \mathbf{x}_j^\top (\mathbf{y} - \mathbf{X}\hat{\theta}) \right| \leq \lambda.$$

La esquina de la penalización permite que un intervalo de gradientes residuales sea compatible con cero. El operador de umbral suave

$$S(a, \lambda) = \text{sign}(a) \max\{|a| - \lambda, 0\}$$

aparece al resolver una coordenada a la vez. Si $\mathbf{r}_{-j} = \mathbf{y} - \sum_{k \neq j} \mathbf{x}_k \theta_k$, la actualización es

$$\theta_j \leftarrow \frac{S(\mathbf{x}_j^\top \mathbf{r}_{-j}/n, \lambda)}{\|\mathbf{x}_j\|_2^2/n}.$$

Con variables muy correlacionadas, Lasso puede escoger una y dejar otra en cero de manera sensible a la muestra. Un cero describe la solución penalizada para ese diseño y ese λ ; no demuestra irrelevancia causal ni ausencia de asociación poblacional.

aspecto	Ridge	Lasso
penalización	suma de cuadrados	suma de valores absolutos
efecto habitual	encogimiento continuo	algunos ceros exactos
columnas correlacionadas	tiende a repartir peso	puede escoger una
cálculo	sistema lineal	algoritmo iterativo, por ejemplo coordenadas
propósito frecuente	estabilidad predictiva	esparsidad y predicción

Como ambas penalizaciones actúan sobre coeficientes, las características deben tener escalas comparables. El intercepto se mantiene separado de la estandarización y de la penalización.

Seleccionar sin consultar prueba

Volvamos al polinomio de grado 15. Después de ajustar `PolynomialFeatures` y `StandardScaler` con entrenamiento, obtenemos:

alpha de Ridge	MSE entrenamiento	MSE validación
0	1.542	3.254
0.0001	1.655	2.552
0.01	1.756	2.317
0.1	1.790	2.274
1	1.848	2.199
10	1.953	2.326
100	2.647	3.396

Ridge reduce la brecha del polinomio de grado 15. Sin embargo, el polinomio de grado 3 obtiene MSE de validación 2.172 y resulta ligeramente mejor entre las configuraciones comparadas. Se fija entonces el procedimiento de grado 3 y se consulta prueba por primera vez: su MSE es 2.103. El baseline, cuya media se calculó solo con entrenamiento, obtiene 5.533.

Esta evaluación no prueba que el grado 3 sea óptimo para toda muestra futura. Sí documenta una comparación reproducible en la que la prueba no participó en la selección.

Regreso a las subzonas de Bogotá

En el caso de Bogotá solo hay 32 filas. Una partición de 24 subzonas para ajuste y 8 para evaluación produce una métrica muy sensible a cuáles subzonas quedaron en cada grupo.

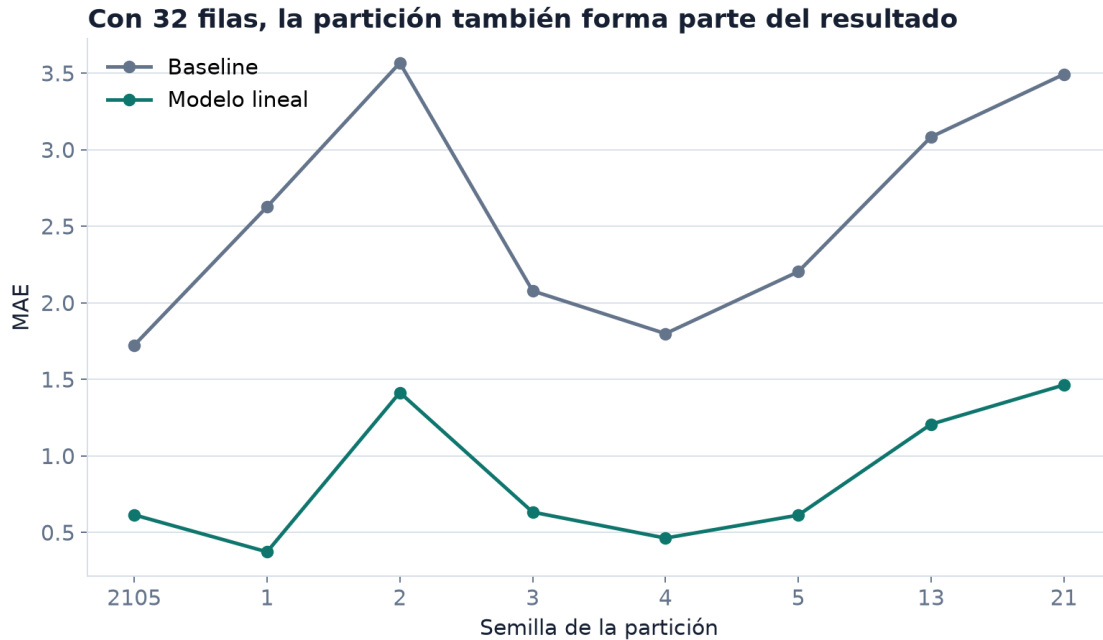


Figura 3: El MAE cambia entre particiones de las 32 subzonas. La semilla registra una fuente de variación; no debe escogerse por producir el menor error.

Escoger la semilla con menor MAE sería otra forma de usar la evaluación para diseñar el resultado. Una comparación responsable fija el mecanismo de partición antes de ver las métricas y muestra su variación. Con tan pocas unidades, la validación cruzada puede aprovechar mejor las filas, pero las unidades siguen siendo agregadas y no están documentadas como una muestra aleatoria de una población más amplia.

Ridge puede estabilizar los coeficientes correlacionados de área, alcobas, baños y parqueaderos. Esa estabilidad resuelve una dificultad del ajuste; no recupera la variación dentro de cada subzona ni la unidad ausente de `precio_reportado`.

Síntesis

Una familia más flexible siempre puede mejorar o conservar el mejor ajuste de entrenamiento. El ejemplo polinomial muestra por qué ese orden no determina el comportamiento fuera de muestra. Entrenamiento ajusta parámetros y transformaciones; validación compara configuraciones; prueba evalúa el procedimiento fijado.

Ridge añade una penalización cuadrática que encoge especialmente las direcciones mal determinadas. Lasso utiliza valores absolutos y puede producir ceros por su condición de subgradiente. En ambos casos, el intercepto no se penaliza y las características se estandarizan con estadísticas aprendidas solo en entrenamiento.

Regularizar, validar y evitar filtración son partes del mismo procedimiento. Una buena métrica de prueba no borra, sin embargo, las restricciones de unidad, muestreo y medición establecidas antes del ajuste.

Problemas integrados

1. Complejidad que mejora el objetivo equivocado

Use `x1` y `target` de `regresion_lineal_sintetica.csv` con las particiones descritas al inicio.

1. Reproduzca la tabla para grados 1, 3, 8 y 15.
2. Grafique cada polinomio dentro y fuera del rango central de entrenamiento.
3. Compare normas de coeficientes y números de condición.
4. Explique por qué el error de entrenamiento debe ser no creciente si los espacios polinomiales están anidados y la solución es exacta.
5. Seleccione un grado con validación y consulte prueba una sola vez.

2. Una tubería con y sin filtración

Construya una variable con faltantes y una categoría poco frecuente.

1. Divida los datos antes de imputar, codificar o estandarizar.
2. Implemente un pipeline que aprenda todas esas transformaciones con entrenamiento.
3. Implemente deliberadamente una versión que las aprenda con toda la tabla.
4. Compare los objetos aprendidos, no solo la métrica final.
5. Identifique cuál información reservada ingresó en cada transformación y por qué una diferencia pequeña en la métrica no vuelve correcto el procedimiento.

3. Ridge en una dirección casi redundante

Use el conjunto sintético y añada $x_4 = x_1 + 0.001 \cdot \text{ruido}$, con semilla fija.

1. Estandarice las características usando solo entrenamiento.
2. Compare valores singulares y coeficientes de OLS con los de Ridge para una rejilla de λ .
3. Verifique la fórmula $\sigma_j / (\sigma_j^2 + n\lambda)$ en las direcciones singulares.
4. Compare pérdida de entrenamiento, validación y norma de coeficientes.
5. Explique qué estabilidad se ganó y qué sesgo se introdujo.

4. Lasso y columnas correlacionadas

Genere dos predictores altamente correlacionados y una respuesta que dependa de ambos.

1. Ajuste Lasso para una secuencia de penalizaciones.
2. Verifique la condición de subgradiente en una coordenada igual a cero.
3. Repita el experimento con diez semillas y registre cuál variable queda activa.
4. Compare predicciones y selección de columnas entre repeticiones.
5. Redacte una conclusión que distinga esparsidad computada, estabilidad y relevancia sustantiva.

5. Validación con 32 subzonas

Use el caso de Bogotá.

1. Defina una tubería con estandarización y Ridge.
2. Compare una rejilla de penalizaciones mediante particiones fijadas o validación cruzada.
3. Incluya el baseline en cada división y conserve resultados por subzona.
4. Muestre la distribución de métricas y la estabilidad de coeficientes.
5. Escriba qué conclusión permiten esas 32 filas y qué población adicional no puede afirmarse sin un diseño de muestreo.

Respuestas seleccionadas

Problema 1. Los MSE de validación para grados 1, 3, 8 y 15 son aproximadamente 2.223, 2.172, 2.379 y 3.254. El mínimo de entrenamiento del grado 15 no justifica escogerlo: la configuración se elige con observaciones que no intervinieron en sus coeficientes.

Problema 3. Si una dirección tiene σ_j pequeño, OLS multiplica la componente correspondiente de $\mathbf{U}^\top \mathbf{y}$ por $1/\sigma_j$. Ridge la multiplica por $\sigma_j/(\sigma_j^2 + n\lambda)$, que permanece acotado y tiende a cero cuando λ crece.

Regresión logística

Probabilidades, pérdida y decisiones en clasificación binaria

El conjunto `clasificacion_binaria_sintetica.csv` contiene 240 observaciones. La respuesta `target` vale 1 o 0 y las entradas disponibles para el modelo son `x1` y `x2`. La columna `slice` identifica dos periodos de generación. En las 180 observaciones del periodo `base`, la proporción de unos es 0.428.

Una regla que siempre predice la clase mayoritaria alcanza accuracy 0.572 sin utilizar ninguna entrada y nunca detecta un positivo. Ese número fija un baseline, pero no responde la pregunta más importante: ¿podemos estimar cómo cambia la probabilidad de $Y = 1$ con $X = (X_1, X_2)$?

El archivo también publica `score_latente` y `probabilidad` para que el mecanismo sintético pueda auditarse. Esas columnas revelan directamente cantidades usadas para generar la respuesta. Incluirlas como predictores convertiría el ejercicio en filtración de información. El modelo se ajustará solo con `x1` y `x2`.

De una clase a una probabilidad condicional

Para una respuesta binaria,

$$Y \mid X = x \sim \text{Bernoulli}(\eta(x)),$$

donde

$$\eta(x) = \mathbb{P}(Y = 1 \mid X = x) = \mathbb{E}[Y \mid X = x].$$

η es una propiedad de la distribución poblacional. La regresión logística no la conoce; construye una aproximación dentro de la familia

$$p_{\theta}(x) = \sigma(\phi(x)^{\top} \theta),$$

con

$$\sigma(z) = \frac{1}{1 + e^{-z}}.$$

La sigmoide es creciente, toma valores estrictamente entre 0 y 1 y satisface $\sigma(0) = 1/2$. Su implementación debe evitar desbordamientos para logits de gran magnitud:

```
import numpy as np

def sigmoid(z):
    z = np.asarray(z, dtype=float)
    output = np.empty_like(z)
    nonnegative = z >= 0
    output[nonnegative] = 1 / (1 + np.exp(-z[nonnegative]))
    exp_z = np.exp(z[~nonnegative])
    output[~nonnegative] = exp_z / (1 + exp_z)
    return output
```

Para un diseño $\mathbf{X} \in \mathbb{R}^{n \times p}$,

$$z = \mathbf{X}\theta, \quad \hat{p} = \sigma(z).$$

La igualdad $\eta(x) = p_\theta(x)$ es un supuesto de especificación, no una consecuencia de aplicar una sigmoide. Por eso diremos *probabilidad estimada* al hablar de la salida ajustada.

Una interpretación mediante odds

La inversa de la sigmoide da

$$\log \left(\frac{p_\theta(x)}{1 - p_\theta(x)} \right) = \phi(x)^\top \theta.$$

Si las demás columnas se mantienen fijas, aumentar una unidad la característica j suma θ_j a los log-odds y multiplica los odds por e^{θ_j} . El cambio en probabilidad no es constante: depende del punto de la sigmoide en el que se evalúa.

La pérdida proviene del modelo Bernoulli

Si una observación tiene probabilidad estimada p_i , su masa Bernoulli es

$$p_i^{y_i} (1 - p_i)^{1-y_i}.$$

Bajo independencia condicional, la verosimilitud de la muestra es el producto de esas masas. Maximizar su logaritmo equivale a minimizar la log-loss promedio

$$J(\theta) = -\frac{1}{n} \sum_{i=1}^n [y_i \log \hat{p}_i + (1 - y_i) \log(1 - \hat{p}_i)].$$

Con $z_i = \phi(x_i)^\top \theta$, la misma cantidad puede escribirse de forma numéricamente estable:

$$J(\theta) = \frac{1}{n} \sum_{i=1}^n [\log(1 + e^{z_i}) - y_i z_i].$$

```
def log_loss_from_logits(y, z):
    y = np.asarray(y, dtype=float)
    z = np.asarray(z, dtype=float)
    return np.mean(np.logaddexp(0.0, z) - y * z)

def log_loss_binary(y, p, eps=1e-15):
    y = np.asarray(y, dtype=float)
    p = np.clip(np.asarray(p, dtype=float), eps, 1 - eps)
    return -np.mean(y * np.log(p) + (1 - y) * np.log(1 - p))
```

Recortar probabilidades evita evaluar $\log(0)$ cuando solo se dispone de probabilidades. Trabajar con logits mediante `logaddexp` es preferible durante el ajuste.

Por qué la log-loss busca la probabilidad condicional

Fijemos x y sea $\eta = \mathbb{P}(Y = 1 \mid X = x)$. El riesgo condicional de reportar $q \in (0, 1)$ es

$$L_x(q) = -\eta \log q - (1 - \eta) \log(1 - q).$$

La diferencia respecto del mínimo es

$$L_x(q) - L_x(\eta) = \text{KL}(\text{Bern}(\eta) \parallel \text{Bern}(q)) \geq 0.$$

La igualdad ocurre en $q = \eta$. La log-loss no solo revisa de qué lado de un umbral queda una observación: compara una distribución Bernoulli completa. Una probabilidad muy segura y equivocada recibe una penalización grande.

Gradiente, Hessiana y convexidad

Para una observación,

$$\frac{d\sigma(z)}{dz} = \sigma(z)(1 - \sigma(z))$$

y la regla de la cadena reduce la derivada de la log-loss respecto del logit a

$$\frac{\partial \ell_i}{\partial z_i} = \hat{p}_i - y_i.$$

Como $\nabla_{\theta} z_i = \phi(x_i)$, al sumar las observaciones se obtiene

$$\nabla J(\theta) = \frac{1}{n} \mathbf{X}^{\top} (\hat{\mathbf{p}} - \mathbf{y}).$$

```
def logistic_gradient(X, y, theta):  
    probabilities = sigmoid(X @ theta)  
    return X.T @ (probabilities - y) / X.shape[0]
```

Sea $\mathbf{W}(\theta)$ la matriz diagonal con entradas

$$W_{ii} = \hat{p}_i(1 - \hat{p}_i).$$

La Hessiana es

$$\nabla^2 J(\theta) = \frac{1}{n} \mathbf{X}^{\top} \mathbf{W}(\theta) \mathbf{X}.$$

Para todo $v \in \mathbb{R}^p$,

$$v^{\top} \nabla^2 J(\theta) v = \frac{1}{n} \sum_{i=1}^n \hat{p}_i(1 - \hat{p}_i) (\phi(x_i)^{\top} v)^2 \geq 0.$$

Convexidad de la log-loss logística. La log-loss empírica de la regresión logística es convexa. Si no hay direcciones no identificadas y los pesos son positivos, es estrictamente convexa en la región correspondiente.

La convexidad no garantiza que el mínimo se alcance en un parámetro finito. Si una recta separa perfectamente las clases, multiplicar sus coeficientes puede llevar las probabilidades hacia 0 y 1 mientras la pérdida se aproxima a cero y $\|\theta\|$ crece sin límite.

Pérdida decreciente con coeficientes crecientes. En datos separables, esta trayectoria no indica que falten iteraciones. Indica que el problema sin penalización no tiene un minimizador finito. Una penalización L2, sin incluir el intercepto, controla esa dirección y produce una solución finita.

Un paso batch de gradiente cuesta $O(np)$. Los métodos de Newton utilizan la Hessiana, cuyo ensamblaje denso cuesta $O(np^2)$ y cuya solución cuesta $O(p^3)$. Las bibliotecas eligen entre métodos de Newton, quasi-Newton, gradiente y coordenadas según el tamaño y la penalización. El objetivo estadístico y el solver son decisiones diferentes.

La probabilidad todavía no es una decisión

Una regla binaria introduce un umbral τ :

$$\hat{y}_i(\tau) = \begin{cases} 1, & \hat{p}_i \geq \tau, \\ 0, & \hat{p}_i < \tau. \end{cases}$$

```
def predict_class(p, threshold=0.5):  
    return (np.asarray(p) >= threshold).astype(int)
```

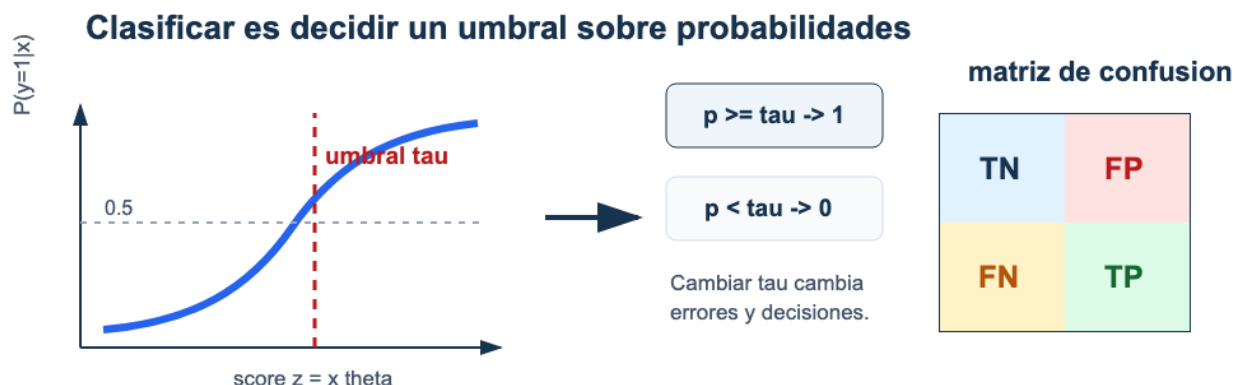


Figura 1: La sigmoide produce probabilidades. El umbral convierte esas probabilidades en clases y modifica la matriz de confusión.

El valor 0.5 no es una ley del modelo. Si un falso negativo cuesta más que un falso positivo, puede ser razonable predecir la clase positiva con una probabilidad menor. Si las probabilidades fueran conocidas y calibradas, con costos C_{FP} y C_{FN} la regla de costo esperado elegiría la clase positiva cuando

$$p \geq \frac{C_{FP}}{C_{FP} + C_{FN}}.$$

En la práctica, p es estimada, los costos pueden ser incompletos y la calibración puede fallar. El umbral se fija con validación y su consecuencia se reporta mediante conteos, no solo mediante una métrica agregada.

Matriz de confusión y métricas

Adoptaremos la orientación habitual:

clase observada	predicho 0	predicho 1
real 0	TN	FP
real 1	FN	TP

```
def confusion_counts(y_true, y_pred):
    y_true = np.asarray(y_true)
    y_pred = np.asarray(y_pred)
    tp = np.sum((y_true == 1) & (y_pred == 1))
    fp = np.sum((y_true == 0) & (y_pred == 1))
    tn = np.sum((y_true == 0) & (y_pred == 0))
    fn = np.sum((y_true == 1) & (y_pred == 0))
    return tp, fp, tn, fn
```

Los cuatro conteos producen

$$\text{accuracy} = \frac{TP + TN}{TP + TN + FP + FN},$$

$$\text{precision} = \frac{TP}{TP + FP}, \quad \text{recall} = \frac{TP}{TP + FN},$$

$$F_1 = 2 \frac{\text{precision recall}}{\text{precision} + \text{recall}}.$$

Cuando un denominador es cero, la métrica necesita una convención explícita; no debe producirse un número silenciosamente.

```
def classification_metrics(y_true, y_pred):
    tp, fp, tn, fn = confusion_counts(y_true, y_pred)
    total = tp + fp + tn + fn
    accuracy = (tp + tn) / total if total else np.nan
    precision = tp / (tp + fp) if (tp + fp) else 0.0
    recall = tp / (tp + fn) if (tp + fn) else 0.0
    f1 = (
        2 * precision * recall / (precision + recall)
        if precision + recall else 0.0
    )
    return {
        "accuracy": accuracy,
        "precision": precision,
        "recall": recall,
        "f1": f1,
    }
```

Accuracy pondera por igual las observaciones y puede ocultar que la clase positiva nunca se detecta. Precision condiciona en las predicciones positivas; recall condiciona en los positivos reales. F_1 combina ambas, pero tampoco incorpora por sí solo un costo, una capacidad de revisión ni la prevalencia de la población de uso.

Un umbral fijado con validación

Usamos únicamente las 180 observaciones del periodo **base**: 108 para entrenamiento, 36 para validación y 36 para prueba, con estratificación y semillas fijas. Una regresión logística ajustada con **x1** y **x2** obtiene log-loss de validación 0.533. Las clases de validación cambian así:

umbral	TN	FP	FN	TP	accuracy	precision	recall	F1
0.3	6	15	1	14	0.556	0.483	0.933	0.636
0.4	11	10	2	13	0.667	0.565	0.867	0.684
0.5	18	3	3	12	0.833	0.800	0.800	0.800
0.6	20	1	7	8	0.778	0.889	0.533	0.667
0.7	20	1	12	3	0.639	0.750	0.200	0.316

Supongamos que el requisito fijado antes de mirar la tabla es alcanzar recall al menos 0.85 y, entre los umbrales que lo cumplen, usar el mayor para limitar falsos positivos. En esta rejilla se escoge $\tau = 0.4$.

```
def choose_threshold_for_recall(y_true, p, min_recall=0.85):
    candidates = np.sort(np.unique(p))
    feasible = []
    for threshold in candidates:
        metrics = classification_metrics(
            y_true, predict_class(p, threshold)
        )
        if metrics["recall"] >= min_recall:
            feasible.append(threshold)
    return max(feasible) if feasible else None
```

Una vez fijado el modelo y el umbral, prueba se consulta una sola vez. Allí se obtienen TN 9, FP 11, FN 0 y TP 16: accuracy 0.694, precision 0.593, recall 1.0 y $F_1 = 0.744$. El umbral 0.5 habría dado mayor accuracy en prueba, pero escogerlo después de ver ese resultado violaría el procedimiento declarado.

La regla mayoritaria alcanza accuracy 0.556 en esta prueba y recall cero. El baseline muestra por qué la métrica primaria debe definirse antes de comparar modelos.

Calibración y alcance de las probabilidades

Un modelo está calibrado si, entre unidades con probabilidad estimada cercana a q , la frecuencia observada de positivos es cercana a q . La calibración es distinta del umbral y también de la capacidad para ordenar casos. Una sigmoide no garantiza calibración fuera de muestra.

En el dataset sintético podemos comparar las probabilidades estimadas con la columna **probabilidad**, porque conocemos el mecanismo generador. Esa comparación es un privilegio del experimento controlado. En una aplicación real, la calibración se estima con observaciones reservadas y conserva incertidumbre.

Las 60 filas del periodo **reciente** no se usarán para reajustar el ejemplo ni para escoger el umbral. Su distribución fue modificada deliberadamente y se retomará en la parte de estrategia para estudiar qué ocurre cuando cambian la prevalencia o las entradas. Mezclarlas aquí ocultaría precisamente el cambio que queremos diagnosticar después.

Síntesis

La regresión logística aproxima la probabilidad condicional de una respuesta Bernoulli mediante una sigmoide aplicada a un predictor lineal. La log-loss proviene de la verosimilitud Bernoulli y es una pérdida propia: su objetivo poblacional es la probabilidad condicional, no solo una etiqueta.

El gradiente conserva la forma matricial $\mathbf{X}^\top(\hat{\mathbf{p}} - \mathbf{y})/n$ y la Hessiana es semidefinida positiva. La convexidad evita mínimos locales espurios, pero la separabilidad puede llevar los coeficientes al infinito si no hay regularización.

El modelo termina en una probabilidad estimada. El umbral añade una decisión y determina la matriz de confusión. Precision, recall, F_1 y accuracy resumen aspectos distintos; ninguna reemplaza la declaración de la clase positiva, los costos y el procedimiento de validación.

Problemas integrados

1. Del mecanismo Bernoulli al gradiente

Use las filas **base** de `clasificacion_binaria_sintetica.csv` y solo **x1**, **x2** como entradas.

1. Calcule la prevalencia y el baseline mayoritario.
2. Derive la log-loss desde la masa Bernoulli.
3. Implemente sigmoide, pérdida desde logits y gradiente.
4. Verifique el gradiente por diferencias finitas en cinco vectores de parámetros.
5. Compare su ajuste con `LogisticRegression` usando la misma penalización o una penalización suficientemente pequeña y documentada.

2. Una probabilidad y dos decisiones

Considere etiquetas $\mathbf{y} = (1, 0, 1, 0, 1, 0)$ y probabilidades $\hat{p} = (0.92, 0.61, 0.55, 0.40, 0.20, 0.10)$.

1. Calcule log-loss sin aplicar umbral.
2. Construya matrices de confusión para $\tau = 0.5$ y $\tau = 0.7$.
3. Calcule accuracy, precision, recall y F_1 .
4. Proponga costos que hagan razonable cada umbral.
5. Explique qué cambió en la decisión y qué permaneció igual en el modelo.

3. Selección de umbral reproducible

Reproduzca las particiones 108/36/36 del periodo **base**.

1. Ajuste estandarización y regresión solo con entrenamiento.
2. Construya la tabla de validación para una rejilla de umbrales.
3. Declare antes de consultar prueba una restricción de recall o una función de costo.
4. Fije el umbral, evalúe prueba una sola vez e incluya los cuatro conteos.
5. Compare con la clase mayoritaria y explique por qué no cambiaría el umbral después de ver prueba.

4. Separabilidad y regularización

Construya dos grupos linealmente separables en $\mathbb{R}^2$.

1. Ejecute gradiente sin penalización y registre pérdida y norma de parámetros.
2. Muestre que las clases permanecen correctas mientras la norma crece.
3. Añada una penalización L2 que no incluya el intercepto.
4. Compare convergencia, probabilidades y frontera para varias intensidades.
5. Explique por qué un minimizador finito es una propiedad distinta de tener accuracy de entrenamiento igual a uno.

Respuestas seleccionadas

Problema 2. Con $\tau = 0.5$ se obtiene $(TP, FP, TN, FN) = (2, 1, 2, 1)$ y las cuatro métricas son aproximadamente 0.667. Con $\tau = 0.7$ se obtiene $(1, 0, 3, 2)$: precision es 1, recall es $1/3$, accuracy es $2/3$ y $F_1 = 0.5$. Las probabilidades y su log-loss no cambian al mover el umbral.

Problema 4. En datos separables, multiplicar por una constante positiva un vector que separa aumenta los márgenes y hace que la log-loss se acerque a cero. La penalización L2 crece cuadráticamente con esa constante y evita que la norma pueda aumentar sin costo.

Problemas integradores de regresión y clasificación

De la muestra y la pérdida a una conclusión evaluada

Los problemas de este capítulo reúnen las ideas de la Parte II. No están separados en preguntas conceptuales, cálculos y programación: cada secuencia exige conectar esos niveles. Los datasets se encuentran en el directorio `datos/` del libro y fueron congelados para que los resultados sean reproducibles.

Una solución completa debe declarar la unidad de observación, las dimensiones de los arreglos, la función que se optimiza y los datos usados para cada decisión. Los resultados numéricos deben acompañarse de un diagnóstico y de una conclusión cuyo alcance coincida con la población descrita.

Síntesis de la parte

Regresión lineal, Ridge y regresión logística pueden escribirse mediante una matriz de diseño $\mathbf{X}$, un vector de parámetros θ y una pérdida empírica. Esa escritura común no vuelve idénticos sus objetivos: mínimos cuadrados aproxima una media condicional, log-loss aproxima una probabilidad condicional y las penalizaciones modifican el estimador para controlar su sensibilidad.

La optimización responde cómo calcular parámetros para un problema fijado. La validación responde cómo escoger entre procedimientos que aprendieron de los datos. Los residuales, las curvas de pérdida, el rango, las métricas y la matriz de confusión examinan fallos diferentes. Ninguno corrige una unidad de observación equivocada, una variable filtrada o una población de uso mal definida.

Problemas integrados

1. Una población conocida y muchas muestras

Considere las 160 filas de `regresion_lineal_sintetica.csv` como una población finita. El mecanismo generador conocido es

$$Y = 4 + 2.5X_1 - 1.2X_2 + 0.7X_3 + \varepsilon, \quad \mathbb{E}[\varepsilon \mid X] = 0.$$

1. Calcule el promedio de `target`, el riesgo cuadrático del baseline constante y el riesgo de la función generadora sobre las 160 filas.
2. Extraiga 500 muestras de tamaño 40. En cada una ajuste un intercepto y las tres variables mediante `np.linalg.lstsq`.

3. Grafique las distribuciones muestrales de los cuatro coeficientes y compare sus centros con $(4, 2.5, -1.2, 0.7)$.
4. Compare, para cada muestra, MSE de ajuste y MSE sobre la población finita.
5. Identifique una muestra cuyo error de ajuste sea menor que el de otra, pero cuyo error poblacional sea mayor.
6. Explique la diferencia entre parámetro poblacional, estimador, estimación y riesgo empírico en este experimento.
7. Indique qué parte del análisis dejaría de estar disponible si las 160 filas fueran una muestra y no una población conocida.

2. Mínimos cuadrados desde tres representaciones

Use

$$x = (0, 1, 2, 3), \quad y = (1.1, 2.9, 5.2, 6.8).$$

1. Obtenga intercepto y pendiente mediante las fórmulas centradas de regresión simple.
2. Construya $\mathbf{X}$ con intercepto y derive las ecuaciones normales.
3. Calcule la solución con `lstsq`; reporte rango, valores singulares, predicciones, residuales y MSE.
4. Verifique $\mathbf{X}^\top \mathbf{r} = \mathbf{0}$ y explique su interpretación geométrica.
5. Añada una tercera columna igual a dos veces la segunda. Encuentre dos vectores de parámetros diferentes que generen las mismas predicciones.
6. Compare la solución de norma mínima de `lstsq` con una solución construida por usted.
7. Explique cuáles afirmaciones dependen del rango completo y cuáles permanecen válidas cuando el diseño es deficiente.

3. Una unidad que cambia la optimización

Use las tres características del conjunto sintético. Expresé `x2` primero en su escala original y luego multiplicada por 1000.

1. Calcule $\kappa_2(\mathbf{X})$, el espectro de $\mathbf{H} = \mathbf{X}^\top \mathbf{X}/n$ y el intervalo estable de tasas en las dos representaciones.
2. Ejecute descenso del gradiente con una misma tasa y documente las dos curvas de pérdida.
3. Estandarice las características, sin transformar el intercepto, y repita el cálculo espectral.
4. Verifique el gradiente por diferencias finitas antes de interpretar la convergencia.
5. Compare el límite con `lstsq` y transforme los coeficientes a las unidades originales.
6. Demuestre que las predicciones coinciden después de deshacer la estandarización.
7. Distinga el efecto de la unidad sobre la realidad medida, los coeficientes, la geometría del algoritmo y la función ajustada.

4. Complejidad, selección y prueba

Use únicamente `x1` para predecir `target`. Construya una partición entrenamiento/validación/prueba con semillas fijadas antes de calcular métricas.

1. Compare grados 1, 3, 8 y 15 mediante un pipeline que incluya `PolynomialFeatures` y `StandardScaler`.
2. Grafique MSE de entrenamiento y validación en función del grado.
3. Para los grados 8 y 15, compare Ridge en una rejilla logarítmica de penalizaciones.
4. Registre MSE, norma de coeficientes y número efectivo de características.
5. Fije una regla de selección que use solo validación y aplíquela sin consultar prueba.
6. Evalúe la configuración seleccionada y el baseline una sola vez en prueba.
7. Repita la partición completa con cinco semillas para estudiar la estabilidad de la selección, sin reemplazar la evaluación final por la semilla más favorable.
8. Redacte una conclusión que separe optimización, selección e incertidumbre por partición.

5. Ridge y Lasso ante predictores correlacionados

Genere, con semilla fija,

$$X_2 = X_1 + 0.02U, \quad Y = 2X_1 + 2X_2 + \varepsilon,$$

donde U y ε son ruidos independientes centrados.

1. Divida los datos y ajuste la estandarización solo con entrenamiento.
2. Compare OLS, Ridge y Lasso sobre rejillas de penalización.
3. Muestre trayectorias de coeficientes y métricas de validación.
4. Para Ridge, relacione el encogimiento con los valores singulares del diseño.
5. Para una coordenada que Lasso deja en cero, verifique la condición de subgradiente.
6. Repita con veinte semillas y registre la frecuencia con la que cada variable queda activa.
7. Compare estabilidad de coeficientes y estabilidad de predicciones.
8. Explique por qué un coeficiente cero no demuestra que la variable carezca de relación con la respuesta.

6. De Bernoulli a regresión logística

Use únicamente las 180 filas `base` de `clasificacion_binaria_sintetica.csv`. No utilice `score_latente`, `probabilidad` ni `slice` como predictores.

1. Identifique la clase positiva, calcule su prevalencia y defina dos baselines.
2. A partir de la masa Bernoulli, derive log-loss, gradiente y Hessiana.
3. Implemente `sigmoid`, `log_loss_from_logits` y `logistic_gradient`.
4. Compruebe el gradiente por diferencias finitas en parámetros aleatorios con semilla fija.
5. Ajuste el modelo por gradiente y compare pérdida, parámetros y predicciones con una implementación de biblioteca cuya penalización esté documentada.
6. Verifique numéricamente que la Hessiana sea semidefinida positiva en varios puntos.

7. Compare las probabilidades estimadas con la columna `probabilidad` solo como auditoría posterior del mecanismo sintético.
8. Explique por qué esa comparación no estaría disponible en una aplicación ordinaria.

7. Un modelo, varios umbrales

Conserve una partición independiente de validación y otra de prueba para el problema anterior.

1. Calcule log-loss de validación sin convertir probabilidades en clases.
2. Para umbrales entre 0.1 y 0.9, construya los cuatro conteos y las métricas.
3. Grafique precision, recall y F_1 en función del umbral.
4. Defina dos escenarios de costos y derive el umbral teórico bajo probabilidades calibradas.
5. Fije con validación un umbral para cada escenario y explique cualquier diferencia respecto del umbral teórico.
6. Evalúe prueba una sola vez para el escenario elegido.
7. Compare contra la clase mayoritaria e incluya siempre su recall.
8. Escriba qué afirmaría sobre la decisión y qué no afirmaría sobre la probabilidad verdadera.

8. Cuando las clases son separables

Construya en $\mathbb{R}^2$ dos grupos que una recta separe perfectamente.

1. Ajuste regresión logística sin penalización durante un número creciente de iteraciones.
2. Registre log-loss, accuracy, margen mínimo y norma de parámetros.
3. Explique por qué accuracy puede estabilizarse en uno mientras la log-loss y los parámetros siguen cambiando.
4. Demuestre que multiplicar una dirección separadora reduce la pérdida sin alcanzar cero con parámetros finitos.
5. Añada Ridge sin penalizar el intercepto y compare las soluciones.
6. Estudie cómo cambia el margen y la calibración aparente al variar la penalización.
7. Distinga existencia de un minimizador, convergencia del solver y desempeño fuera de muestra.

9. Un análisis completo de las subzonas de Bogotá

Use `vivienda_nueva_bogota_subzonas.csv`.

1. Reconstruya el diccionario de las siete columnas y documente la unidad de observación. No invente la unidad de `precio_reportado`.
2. Formule una pregunta descriptiva y una predictiva que tengan a la subzona como unidad.
3. Compare baseline, OLS y Ridge mediante un procedimiento de validación fijado antes de ver resultados.
4. Mantenga estandarización, ajuste y selección dentro de un pipeline.
5. Reporte métricas por partición, residuales, rango, valores singulares y estabilidad de coeficientes.
6. Identifique una subzona cuyo residual merezca revisar la documentación, sin eliminarla automáticamente.

7. Explique por qué una mejora predictiva entre subzonas no describe viviendas individuales ni establece causalidad.
8. Escriba una conclusión de no más de 180 palabras con resultado, baseline, variación observada y dos limitaciones concretas.

10. Diseñar una investigación reproducible

Elija un conjunto tabular diferente de los tres casos conductores.

1. Defina unidad, población, salida, momento de predicción y uso previsto.
2. Justifique si el problema es regresión o clasificación y cuál objeto condicional intenta aproximar.
3. Declare un baseline, una métrica y una regla de partición antes del ajuste.
4. Implemente una función central desde cero y compruébela contra una biblioteca.
5. Compare al menos una decisión de complejidad o regularización con validación.
6. Incluya un diagnóstico de optimización y otro de comportamiento fuera de muestra.
7. Conserve semillas, versiones, nombres de columnas y resultados necesarios para reproducir el análisis.
8. Redacte una conclusión que incluya una afirmación respaldada, una limitación y la próxima comprobación necesaria.

Respuestas seleccionadas

Problema 1. En las 160 filas, el promedio de `target` es aproximadamente 4.121. El MSE del baseline constante es 5.277 y el de la función generadora es 0.289. El segundo valor no es cero porque la respuesta incluye ruido incluso condicionado en las entradas.

Problema 2. La solución es $\hat{\theta} = (1.09, 1.94)^\top$, con predicciones (1.09, 3.03, 4.97, 6.91), residuales (0.01, -0.13, 0.23, -0.11) y MSE 0.0205. La suma de residuales y su producto con la columna de x son cero salvo redondeo.

Problema 3. Al multiplicar `x2` por 1000, el número de condición del diseño es aproximadamente 1311 y el límite de paso es 1.68×10^{-6} . Después de estandarizar, esos valores son aproximadamente 2.16 y 1.207. La unidad altera los parámetros y la trayectoria, pero las predicciones pueden transformarse de manera equivalente.

Problema 6. La Hessiana tiene la forma

$$\frac{1}{n} \mathbf{X}^\top \mathbf{W} \mathbf{X}, \quad W_{ii} = \hat{p}_i(1 - \hat{p}_i) \geq 0.$$

Por ello $v^\top \mathbf{X}^\top \mathbf{W} \mathbf{X} v / n = \|\mathbf{W}^{1/2} \mathbf{X} v\|_2^2 / n \geq 0$ para todo v .

Problema 9. Una conclusión admisible se refiere a asociaciones y predicción entre las 32 subzonas incluidas. No puede convertirse en una conclusión sobre viviendas individuales: las filas son agregadas y el archivo no conserva la distribución interna de cada subzona.

Apéndice Módulo 1: formulario y respuestas seleccionadas

Resumen de fórmulas, chequeos y verificación

Cómo usar este apéndice

Este apéndice resume las fórmulas y chequeos del primer bloque matemático del curso: regresión, optimización, regularización y clasificación binaria. Úsalo para verificar derivaciones, notebooks y preparación del Parcial 1.

No memorices fórmulas aisladas. Para cada objeto, verifica forma, significado, hipótesis y el lugar que ocupa en el pipeline de modelación.

Notación mínima

Símbolo	Significado
$X \in \mathbb{R}^{n \times p}$	matriz de diseño
$y \in \mathbb{R}^n$	variable respuesta o etiqueta binaria
$\theta \in \mathbb{R}^p$	vector de parámetros
$\hat{y}$	predicción de regresión
$\hat{p}$	probabilidad predicha en clasificación binaria
λ	intensidad de regularización
τ	umbral de clasificación
TP, TN, FP, FN	conteos de matriz de confusión

Formulario mínimo

Regresión lineal

Predicción:

$$\hat{y} = X\theta.$$

Residual:

$$r = y - X\theta.$$

MSE:

$$\text{MSE}(\theta) = \frac{1}{n} \|X\theta - y\|_2^2.$$

Pérdida escalada:

$$J(\theta) = \frac{1}{2n} \|X\theta - y\|_2^2.$$

Gradiente:

$$\nabla J(\theta) = \frac{1}{n} X^\top (X\theta - y).$$

Ecuación normal:

$$X^\top X\theta = X^\top y.$$

Solución si $X^\top X$ es invertible:

$$\theta^* = (X^\top X)^{-1} X^\top y.$$

Descenso del gradiente

Actualización:

$$\theta^{(t+1)} = \theta^{(t)} - \alpha \nabla J(\theta^{(t)}).$$

Para MSE:

$$\theta^{(t+1)} = \theta^{(t)} - \alpha \frac{1}{n} X^\top (X\theta^{(t)} - y).$$

Estandarización:

$$z_j = \frac{x_j - \mu_j}{s_j}.$$

Ridge

Objetivo:

$$J_{\text{ridge}}(\theta) = \frac{1}{2n} \|X\theta - y\|_2^2 + \frac{\lambda}{2} \|\theta_{1:p}\|_2^2.$$

Gradiente:

$$\nabla J_{\text{ridge}}(\theta) = \frac{1}{n} X^\top (X\theta - y) + \lambda \tilde{\theta}.$$

Solución sin penalizar intercepto:

$$\theta^* = (X^\top X + n\lambda D)^{-1} X^\top y,$$

donde $D_{00}=0$ y $D_{jj}=1$ para $j>0$.

Lasso

Objetivo:

$$J_{\text{lasso}}(\theta) = \frac{1}{2n} \|X\theta - y\|_2^2 + \lambda \|\theta_{1:p}\|_1.$$

Regresión logística

Sigmoide:

$$\sigma(z) = \frac{1}{1 + e^{-z}}.$$

Probabilidad:

$$\hat{p} = \sigma(X\theta).$$

Log-loss:

$$J(\theta) = -\frac{1}{n} \sum_{i=1}^n [y_i \log(\hat{p}_i) + (1 - y_i) \log(1 - \hat{p}_i)].$$

Gradiente:

$$\nabla J(\theta) = \frac{1}{n} X^\top (\hat{p} - y).$$

Clasificación por umbral:

$$\hat{y}_i = 1\{\hat{p}_i \geq \tau\}.$$

Métricas

Accuracy:

$$\frac{TP + TN}{TP + TN + FP + FN}.$$

Precision:

$$\frac{TP}{TP + FP}.$$

Recall:

$$\frac{TP}{TP + FN}.$$

F1:

$$2 \frac{\text{precision} \cdot \text{recall}}{\text{precision} + \text{recall}}.$$

Patrones de código

```
def add_intercept(X):  
    X = np.asarray(X, dtype=float)  
    return np.hstack([np.ones((X.shape[0], 1)), X])
```

```
def mse_loss(X, y, theta):  
    residual = X @ theta - y  
    return np.mean(residual ** 2)
```

```
def mse_gradient(X, y, theta):  
    return (X.T @ (X @ theta - y)) / X.shape[0]
```

```
def ridge_closed_form(X, y, lam):  
    p = X.shape[1]  
    D = np.eye(p)  
    D[0, 0] = 0  
    return np.linalg.solve(X.T @ X + X.shape[0] * lam * D, X.T @ y)
```

```
def sigmoid(z):
    z = np.asarray(z, dtype=float)
    out = np.empty_like(z)
    pos = z >= 0
    out[pos] = 1 / (1 + np.exp(-z[pos]))
    exp_z = np.exp(z[~pos])
    out[~pos] = exp_z / (1 + exp_z)
    return out
```

Checklist antes de entregar

- El notebook corre desde cero.
- Las funciones no dependen de variables globales ocultas.
- Las formas de matrices y vectores son consistentes.
- El preprocesamiento se ajusta solo con entrenamiento.
- Hay baseline.
- Hay métrica adecuada.
- Hay interpretación breve.
- La conclusión menciona al menos una limitación.

Respuestas seleccionadas

Estas respuestas no reemplazan el desarrollo. Sirven para verificar dirección.

Capítulo 1

Ejercicio 1.1. Si X tiene forma $(120, 4)$, entonces `theta` debe tener forma $(4,)$ o $(4, 1)$ para que `X @ theta` produzca 120 predicciones.

Ejercicio 1.2. La columna de unos convierte el intercepto en un coeficiente más: `theta_0 * 1`.

Ejercicio 1.6. Para

$$X = \begin{bmatrix} 1 & 0 \\ 1 & 1 \\ 1 & 2 \end{bmatrix}, y = \begin{bmatrix} 2 \\ 2 \\ 5 \end{bmatrix},$$

se obtiene:

$$X^T X = \begin{bmatrix} 3 & 3 \\ 3 & 5 \end{bmatrix}, \quad X^T y = \begin{bmatrix} 9 \\ 12 \end{bmatrix}.$$

Capítulo 2

Ejercicio 2.2. Usamos $\theta - \alpha \cdot \text{grad}$ porque el gradiente apunta hacia mayor aumento de la pérdida.

Ejercicio 2.4. Escalar ayuda porque reduce diferencias extremas de curvatura entre direcciones. El descenso hace pasos más directos y estables.

Ejercicio 2.8. Una curva que sube y baja violentamente sugiere una tasa de aprendizaje demasiado grande o un problema de escala.

Capítulo 3

Ejercicio 3.2. No se escoge λ con prueba porque entonces prueba deja de ser una evaluación final independiente.

Ejercicio 3.8. Para $\theta=(3,4,-1)$, sin intercepto:

$$\|\theta_{1:p}\|_2^2 = 4^2 + (-1)^2 = 17, \quad \|\theta_{1:p}\|_1 = 4 + 1 = 5.$$

Ejercicio 3.16. Preferiría $\lambda=0.1$ si la meta es generalizar: aumenta el error de entrenamiento, pero reduce mucho el error de validación.

Capítulo 4

Ejercicio 4.6.

$$\sigma(0) = 0.5, \quad \sigma(2) \approx 0.881, \quad \sigma(-2) \approx 0.119.$$

Ejercicio 4.7. Si TP=40, FP=10, TN=80, FN=20:

$$\text{accuracy} = 0.80, \quad \text{precision} = 0.80, \quad \text{recall} = 0.667.$$

F1 es aproximadamente 0.727.

Ejercicio 4.16. Accuracy alta y recall bajo puede indicar clases desbalanceadas: el modelo acierta muchos negativos, pero falla positivos.

Tema	Ejercicios recomendados
------	-------------------------

Guía de estudio para el Parcial 1

Prioriza estos ejercicios si tienes poco tiempo:

Tema	Ejercicios recomendados
Regresión lineal	Cap. 1: 6, 7, 10, 14
Gradiente	Cap. 2: 6, 9, 13, 14
Validación	Cap. 3: 2, 6, 10, 16
Regularización	Cap. 3: 7, 8, 11, 13
Logística	Cap. 4: 6, 7, 11, 12
Umbrales	Cap. 4: 14, 15, 16

Para cerrar, escribe una página de resumen con:

1. una fórmula que todavía te cuesta;
2. un error común que ya sabes evitar;
3. una métrica que usarías en tu proyecto;
4. una pregunta concreta para clase o monitoría.

Lab 1: Regresión lineal y descenso del gradiente

Mínimos cuadrados desde cero con revisión manual mínima

Propósito

Este laboratorio verifica que puedes pasar de la formulación matemática de regresión lineal a una implementación vectorizada en NumPy. También compara dos rutas hacia el mismo ajuste: mínimos cuadrados con `np.linalg.lstsq` y descenso del gradiente. Las ecuaciones normales se derivan en el libro, pero no se forma $X^\top X$ para resolver numéricamente el problema.

Además debes distinguir el riesgo cuadrático poblacional $R_P(\theta) = \mathbb{E}[(Y - X^\top \theta)^2]$ de su versión empírica, reconocer que $\hat{\theta}$ es un estimador aleatorio y cuantificar el error estándar de una media de pérdidas.

La pregunta técnica central es:

¿La implementación desde cero produce las mismas predicciones y pérdidas que una implementación de referencia, y puedes explicar cómo sabes que el algoritmo convergió?

Qué debes entregar

Entrega en Gradescope un único notebook con el nombre:

```
lab01_regresion_gradiente.ipynb
```

No cambies nombres de funciones, firmas, imports ni estructura general del notebook. El autograder busca funciones por nombre. GitHub puede usarse para guardar versiones, pero un **push** no reemplaza la entrega en Gradescope.

La interpretación se escribe únicamente en la celda Markdown marcada **Tu respuesta**. No agregues la respuesta al enunciado ni incluyas nombre, correo o código estudiantil dentro de esa celda.

Flujo recomendado

1. Ejecuta el notebook completo una vez para ver qué está incompleto.
2. Completa primero las funciones pequeñas: `add_intercept`, `predict_linear`, `mse_loss`.
3. Implementa `normal_equation` con `np.linalg.lstsq`. El nombre se conserva para no romper el contrato del autograder.
4. Implementa `mse_gradient` y verifica dimensiones.
5. Implementa `standard_error_mean` con desviación muestral.
6. Completa `probability_answers` separando población, estimador y estimación.
7. Implementa `gradient_descent` guardando historia de pérdida.
8. Compara tus resultados con `sklearn.linear_model.LinearRegression`.
9. Escribe la interpretación final en lenguaje técnico breve.
10. Reinicia el kernel y ejecuta todo de arriba hacia abajo antes de entregar.

Lo que autocalifica el notebook

Componente	Evidencia esperada	Calificación
Intercepto	X aumenta una columna de unos sin alterar filas	Automática
Pérdida	valor de $1/(2n) * \text{sum}(\text{residual}^2)$ correcto	Automática
Error estándar	<code>sd(values, ddof=1) / sqrt(m)</code> con validación de entrada	Automática
Objetos probabilísticos	riesgo esperado, estimador aleatorio y papel de test	Automática
Mínimos cuadrados	coincide con <code>lstsq</code> , incluso si la matriz pierde rango	Automática
Gradiente	coincide con fórmula matricial y tolerancias	Automática
Descenso del gradiente	pérdida decrece y parámetros son consistentes	Automática
Comparación de referencia	resultado compatible con <code>lstsq</code> en datos alternos	Automática

Parte manual

Incluye una respuesta de 80 a 140 palabras en la celda independiente:

¿Qué evidencia muestra que el descenso del gradiente convergió, qué cantidad poblacional intenta estimar el MSE de test y cuál es una fuente de incertidumbre?

Rúbrica manual:

	Nivel	Criterio
	0	No responde o la respuesta no usa evidencia del notebook
	1	Menciona pérdida o limitación, pero sin conectar con comparación/baseline
	2	Conecta curva de pérdida y referencia, distingue riesgo esperado/empírico y nombra una fuente concreta de incertidumbre

Criterios de calidad

- Todas las funciones son vectorizadas.
- No hay valores hardcodeados para pasar un ejemplo específico.
- Las formas de `X`, `y`, `theta` y predicciones son consistentes.
- La historia de pérdida permite diagnosticar convergencia.
- El modelo se compara contra un baseline y contra una implementación de referencia.
- La interpretación final menciona una limitación.
- La interpretación separa θ_P , $\hat{\theta}$ y el vector calculado en esta muestra.

Errores frecuentes

1. Agregar intercepto dos veces.
2. Devolver MSE cuando la función pide $1/(2n)$ por suma de cuadrados.
3. Formar $X^T X$ para resolver cuando `np.linalg.lstsq` evita ese paso.
4. Confundir residual $y - \hat{y}$ con $\hat{y} - y$ dentro del gradiente.
5. No guardar la pérdida en cada iteración.
6. Usar una tasa de aprendizaje que diverge y no revisar la curva.
7. Entregar sin reiniciar kernel y ejecutar todo.

Antes de entregar

Ejecuta este control mental:

```
¿Mi notebook corre completo?  
¿Cada función devuelve la forma esperada?  
¿La pérdida del gradiente baja?  
¿La solución por gradiente se acerca a la obtenida con `lstsq`?  
¿La respuesta manual cita resultados concretos y no solo una opinión?
```

Ejecuta además el validador público antes de cargar el archivo:

```
python validar_entrega.py lab01_regresion_gradiente.ipynb
```

Lab 2: Regularización, validación y clasificación

Ridge, Lasso, logística, métricas y umbrales

Propósito

Este laboratorio evalúa si puedes controlar complejidad, evitar leakage y justificar una regla de clasificación con métricas apropiadas. Conecta tres ideas del Módulo 1:

1. regularización para estabilizar modelos;
2. validación para escoger hiperparámetros;
3. umbrales para convertir probabilidades en decisiones.

La clasificación se formula como $Y \mid X = x \sim \text{Bernoulli}(p(x))$. La probabilidad producida por el modelo es una estimación de $p(x)$, y las métricas de validación o prueba son estimadores aleatorios del desempeño bajo una población objetivo.

Qué debes entregar

Entrega en Gradescope un único notebook con el nombre:

```
lab02_regularizacion_clasificacion.ipynb
```

No cambies nombres ni firmas. El autograder revisa las funciones con datos alternos y también verifica las respuestas estructuradas sobre el protocolo de evaluación. GitHub es un espacio opcional de trabajo; la entrega oficial es el archivo cargado en Gradescope.

Escribe la recomendación únicamente en la celda Markdown marcada **Tu respuesta**. No edites el enunciado ni incluyas datos personales en esa celda.

Flujo recomendado

1. Lee el enunciado completo antes de programar.
2. Separa datos antes de ajustar cualquier transformación.
3. Implementa `standardize_from_train` y ajusta el escalamiento solo con entrenamiento.
4. Implementa Ridge con la convención $X^\top X + \alpha P$, sin penalizar intercepto.
5. Compara Ridge y Lasso con `select_regularization_alpha`, usando validación.
6. Implementa funciones logísticas: `sigmoid`, `log_loss_binary`.
7. Calcula métricas con una matriz de confusión.
8. Implementa `bernoulli_standard_error` para una media de indicadores.
9. Elige el umbral en validación; usa prueba una sola vez al final.
10. Completa `protocol_answers`, incluido modelo Bernoulli y carácter aleatorio de validación.
11. Reinicia kernel y ejecuta todo antes de entregar.

Lo que autocalifica el notebook

Componente	Evidencia esperada	Calificación
Escalamiento	parámetros ajustados solo con entrenamiento	Automática
Ridge	convención numérica correcta sin penalizar intercepto	Automática
Lasso/Ridge	selección por error de validación con datos alternos	Automática
Sigmoide y log-loss	implementación vectorizada y estable	Automática
Métricas	accuracy, precision, recall y F1 correctos	Automática
Incertidumbre	error estándar <i>plug-in</i> de una proporción y semántica poblacional	Automática
Umbral	selección consistente con objetivo de recall	Automática
Protocolo	escalamiento, hiperparámetros, umbral y prueba con papeles correctos	Automática

Parte manual

Incluye una respuesta de 100 a 160 palabras en la celda independiente:

¿Qué modelo o umbral recomendarías, qué cantidad poblacional intenta estimar la métrica reportada y qué incertidumbre comunicarías antes de usarlo?

Rúbrica manual:

	Nivel	Criterio
	0	No hay decisión técnica clara
	1	Hay decisión, pero sin métrica, baseline o limitación
	2	La decisión usa métrica y baseline, distingue probabilidad estimada de verdadera y comunica incertidumbre y alcance

Criterios de calidad

- No hay leakage entre entrenamiento, validación y prueba.
- La métrica elegida corresponde al problema.
- La regularización se interpreta como control de complejidad, no como magia.
- Los umbrales se justifican con costos de FP/FN o una restricción explícita.
- La recomendación final distingue resultado numérico y alcance aplicado.

Errores frecuentes

1. Escalar todo el dataset antes de separar.
2. Escoger `lambda` por error de entrenamiento.
3. Usar prueba repetidamente para tomar decisiones.
4. Penalizar el intercepto en Ridge sin intención.
5. Implementar sigmoide de forma que haga overflow.
6. Dividir por cero en precision o recall.
7. Elegir umbral 0.5 por costumbre sin discutir costos.

Antes de entregar

```
¿Separé datos antes de ajustar transformaciones?  
¿Elegí hiperparámetros con validación?  
¿Reporté una métrica alineada con la decisión?  
¿Mi umbral tiene justificación?  
¿La parte manual comunica una limitación real?
```

Antes de cargar el archivo, ejecuta:

```
python validar_entrega.py lab02_regularizacion_clasificacion.ipynb
```

Parcial 1 autograded

Probabilidad aplicada, regresión, optimización, regularización y clasificación

Alcance

Este parcial evalúa el Módulo 1. Combina implementación, resultados numéricos, razonamiento matemático estructurado e interpretación breve.

Temas incluidos:

1. población, muestra, estimador, estimación y riesgo;
2. media condicional y modelo de ruido en regresión lineal;
3. forma matricial, pérdida MSE y gradiente;
4. descenso del gradiente y regularización Ridge;
5. modelo Bernoulli condicional, regresión logística y log-loss;
6. métricas como estimadores, error estándar y umbrales de clasificación.

Reglas

- No cambies nombres de funciones.
- No uses internet.
- No hardcodees resultados.
- No borres celdas del notebook.
- Ejecuta todo antes de entregar.
- La parte manual se califica con rúbrica corta.

Estructura

Parte	Tema	Tipo de calificación
1	Regresión lineal: objetivo poblacional, predicción, pérdida y gradiente	Autograded
2	Optimización: descenso del gradiente	Autograded
3	Regularización: Ridge y selección de hiperparámetro	Autograded
4	Clasificación: Bernoulli, sigmoide, log-loss, métricas, error estándar y umbral	Autograded
5	Matemática probabilística estructurada	Autograded
6	Explicación técnica breve	Manual

Qué espera el autograder

El autograder revisa:

- formas de salida;
- tolerancias numéricas;
- comportamiento con datos alternos;
- ausencia de hardcoding simple;
- respuestas estructuradas cuando correspondan.

No asumas que pasar un ejemplo visible garantiza pasar todas las pruebas.

Parte manual

Responde:

Si un modelo mejora el baseline por poco, ¿qué cantidad poblacional intenta estimar esa mejora, qué incertidumbre tiene la comparación y qué evidencia adicional pedirías antes de recomendarlo?

Rúbrica:

	Puntaje	Criterio
	0	No responde o la respuesta no es técnica
	5	Menciona una evidencia aislada, como otra métrica o más datos
	10	Identifica cantidad poblacional, variabilidad/validación, análisis de errores y alcance de uso

Preparación recomendada

Antes del parcial:

1. Resuelve los ejercicios seleccionados del apéndice.
2. Implementa desde cero `mse_gradient`, `ridge_closed_form`, `sigmoid` y métricas.
3. Revisa una curva de pérdida y escribe un diagnóstico.
4. Explica precision y recall con una matriz de confusión.
5. Distingue probabilidad condicional verdadera y probabilidad estimada.
6. Calcula el error estándar *plug-in* de una proporción.
7. Practica una recomendación técnica de máximo 120 palabras.

Errores que cuestan puntos

1. Cambiar firmas de funciones.
2. Entregar con celdas sin ejecutar.
3. Usar variables globales ocultas.
4. Hardcodear outputs de ejemplos visibles.
5. Confundir MSE con pérdida escalada $1/(2n)$.
6. Usar prueba para seleccionar hiperparámetros.
7. Reportar accuracy sin mirar clase positiva.

Guía de preparación Parcial 1

Probabilidad aplicada, regresión, gradiente, regularización y clasificación

Alcance

El Parcial 1 evalúa el Módulo 1 completo:

1. Población, muestra, estimador, estimación, riesgo e incertidumbre.
2. Media condicional y modelo de ruido en regresión.
3. Regresión lineal múltiple y ecuación normal.
4. Descenso del gradiente, escalamiento y condicionamiento.
5. Regresión polinomial, overfitting, Ridge, Lasso y validación.
6. Modelo Bernoulli, regresión logística y log-loss.
7. Métricas como estimadores, error estándar y umbrales.

Formato

Parte	Peso	Tipo
Código autograded	70 %	Notebook
Matemática estructurada	20 %	Respuestas numéricas o selección
Explicación corta	10 %	Rúbrica 0/1/2

Lo que debes poder hacer

- Escribir $\mathbf{y_hat} = \mathbf{X} @ \mathbf{theta}$.
- Distinguir θ_P , $\hat{\theta}(D_m)$ y su valor calculado.
- Explicar riesgo esperado frente a riesgo empírico.
- Identificar la media condicional como objetivo bajo pérdida cuadrática.
- Calcular MSE.
- Derivar e implementar el gradiente de MSE.
- Ejecutar descenso del gradiente.
- Explicar efecto de escalamiento.
- Separar entrenamiento, validación y prueba.
- Comparar Ridge/Lasso.
- Implementar sigmoide y log-loss.
- Interpretar matriz de confusión.
- Justificar una métrica según una decisión.
- Calcular e interpretar el error estándar de una media o proporción.

Errores que cuestan puntos

- Usar datos de prueba para elegir hiperparámetros.
- Reportar accuracy sin baseline.
- Penalizar el intercepto sin justificar.
- Cambiar firmas de funciones del notebook.
- Hardcodear respuestas numéricas.
- Entregar código que solo funciona para un dataset específico.

Estrategia de estudio

1. Resuelve los ejercicios del libro.
2. Rehaz los notebooks de clase sin mirar solución.
3. Ejecuta los laboratorios desde cero.
4. Explica cada métrica con una frase técnica.
5. Practica derivaciones cortas: media condicional, MSE, Ridge y log-loss.
6. Clasifica afirmaciones como poblacionales, muestrales o algorítmicas.

Proyecto durante el Módulo 1

De una idea a un baseline reproducible

Objetivo

Durante el Módulo 1 el proyecto pasa de explorar datos a fijar una pregunta y producir una primera referencia cuantitativa. Al terminar la Semana 6, el equipo debe tener un dataset viable, un protocolo de evaluación y un baseline que otra persona pueda ejecutar.

Especificación vigente

Consulta [Proyecto final](#) para nombres de tareas en Bloque Neón, reglas comunes, plantillas y rúbricas. Las cuatro entregas de este módulo se hacen en **Bloque Neón > Contenido > Proyecto final**.

Entregas del módulo

Hito	Fecha límite	Archivo	Puntos
P01 Lista corta de datos	2026-08-13, 23:59	grupo##_hito02_datasets.pdf, máximo 2 páginas	5
P02 Prepropuesta	2026-08-20, 23:59	grupo##_hito03_prepropuesta.pdf, máximo 2 páginas	10
P03 Propuesta aprobable	2026-08-27, 23:59	grupo##_hito04_propuesta.pdf, máximo 4 páginas	20
P04 Baseline	2026-09-10, 23:59	grupo##_hito06_baseline.zip	20

Todas las horas son hora de Bogotá (UTC-5). Una sola persona del grupo sube el archivo y verifica que Bloque Neón muestre la entrega como recibida.

Qué debe resolver cada entrega

P01: lista corta

Compara 2 o 3 datasets por fuente, licencia, unidad de observación, tamaño, objetivo, variables, acceso y riesgo. Cierra con una opción preferida. Una lista de enlaces sin comparación no cumple el propósito.

P02: prepropuesta

Formula pregunta, usuario, población, muestra, unidad, estimando, métrica tentativa, baseline y limitación. Usa la [plantilla del kit](#).

P03: propuesta aprobable

Fija el protocolo antes de observar resultados finales: partición, métrica principal, baseline, modelos candidatos, transformaciones, incertidumbre y regla de selección. La respuesta docente será **Aprobada**, **Aprobada con cambios** o **Reformular**.

P04: baseline

El .zip debe incluir:

```
grupo##_hito06_baseline/  
  README.md  
  baseline.ipynb  
  data/README.md  
  artifacts/metrics.json
```

El notebook ejecuta de arriba hacia abajo y muestra partición, baseline, métrica, resultado, limitación y siguiente experimento.

Qué cuenta como baseline

Un baseline es simple, reproducible y relevante:

- regresión: media, mediana o modelo lineal sencillo;
- clasificación: clase mayoritaria o regla transparente;
- recomendación o no supervisado: popularidad, asignación simple o referencia explícita.

El baseline debe usar la misma partición y métrica que los modelos posteriores. No se selecciona con el test set.

Revisión rápida antes de enviar

1. ¿Existe la fuente y se entiende la licencia?
2. ¿Una fila y la población están definidas por separado?
3. ¿El estimando responde a la pregunta?
4. ¿La métrica expresa el error relevante?
5. ¿El baseline se puede reconstruir desde el archivo entregado?
6. ¿El grupo reconoce una limitación concreta?